

Provably Correct Post-Update Incremental Maintenance of RDB2RDF Views (Extended Version)

Anonymized Author

Anonymized Institution

Abstract. A strategy for exposing relational data as part of a knowledge graph is to define RDB2RDF views that map relational tuples to RDF triples using transformation rules. These views are often materialized in RDF stores to improve query performance and data availability, a process that requires incremental mechanisms to maintain them during updates. A key problem in incremental maintenance is computing the changesets to apply to a view, which traditionally depend on the pre-update state of the database. The central contribution of this paper is a formal model of transformation rules that yields provably correct changesets operating directly on the post-update state. The second contribution is to propose an operational architecture for incremental maintenance of RDB2RDF views that realizes the formal model. The architecture presupposes that the view is defined by R2RML statements and features AFTER triggers and a maintenance worker to implement transformation rules generated from the R2RML statements. The third contribution is a feasibility study of the operational architecture based on an RDB2RDF view, defined by R2RML statements, on a version of the MusicBrainz database.

Keywords: LLM SQL Code Generation · Semantic View · Enterprise Knowledge Graph · Formal Specification.

1 Introduction

Enterprise Knowledge Graphs (EKGs) have emerged as a powerful paradigm for integrating heterogeneous enterprise data sources into a unified semantic layer. By linking structured and semi-structured data via an ontology, EKGs enable applications to query and analyze enterprise information in a semantically rich, integrated manner. In many real-world deployments, the underlying enterprise data remains stored in relational databases, while the knowledge graph provides a semantic access layer on top of these sources.

A common strategy for exposing relational data as part of a knowledge graph is the definition of *RDB2RDF views*, which map relational tuples into RDF triples using transformation rules. These views are often materialized in RDF stores to improve query performance and data availability. However, when the relational source is frequently updated, the materialized RDB2RDF view must

be continuously synchronized with the underlying database. Recomputing the entire RDB2RDF view after each update is typically impractical, motivating the need for *incremental maintenance techniques* that compute only the changes to the view.

A well-established approach for incremental view maintenance is to compute *changesets*, represented as a pair $\langle \Delta^-(u), \Delta^+(u) \rangle$, capturing the triples removed from and added to the RDB2RDF view after a relational update u . Prior work [3] has shown that, for entity-preserving RDB2RDF statements, it is sufficient to identify the *pivot tuples* whose RDF contributions may be affected by the update and recompute their associated RDF state.

The formal framework introduced in [3] provides a rigorous method for computing such changesets based on transformation rules over relational schemas. In particular, in this framework, the deletion component $\Delta^-(u)$ is defined with respect to the pre-update database state. Conceptually, this definition is correct, as it captures precisely the RDF triples that must be removed from the view. However, it introduces a practical limitation when implementing the framework with database triggers, namely, relying on the pre-update database state.

The central contribution of this paper is to challenge this assumption by introducing a formal model that yields provably correct changesets operating directly on the post-update state. It shows that, for a transformation rule without multiple occurrences of the same relation, both deletion and insertion components can be derived using only the post-update state, the transition tables, and the pre-update view state. For a transformation rule with multiple occurrences of the same relation R , it captures the impact of deletions and insertions on R directly in the evaluation of the transformation rule, still without using the pre-update database state.

The second contribution of the paper is to propose an operational architecture for incremental maintenance of RDB2RDF views that realizes the formal model. The architecture decouples SPARQL computation from relational transactions, thereby avoiding the execution of SPARQL queries inside relational transactions and isolating the triplestore. The architecture presupposes that the RDB2RDF view is defined by R2RML statements, which requires a preprocessing step to translate such statements into transformation rules. The paper proposes implementing this preprocessing step using an LLM prompted with the formal model. Then, the paper discusses how to translate the transformation rules into AFTER triggers, again using an LLM prompted with the formal model. Taken together, this means that, given an RDB2RDF view defined by R2RML statements, the underlying relational database can be equipped with AFTER triggers that, combined with the other (fixed) components of the architecture, incrementally maintain the RDB2RDF view.

The third contribution of the paper is a feasibility study of the operational architecture. The study evaluates the complete implementation cycle for an RDB2RDF view built on a version of the MusicBrainz database. The database features a schema with 374 relations that encompass information about artists,

release groups, etc. The RDB2RDF view uses the *Music Ontology* vocabulary and is defined by 57 R2RML statements.

To summarize, existing incremental RDB2RDF maintenance formulations are difficult to realize using statement-level AFTER triggers because their deletion changesets depend on the pre-update relational state, which is no longer directly available when such triggers execute. The proposed formal model removes this mismatch by reformulating the computation of changesets in terms of the post-update relational state, the transition tables, and the pre-update RDF view. Equally important, the resulting changesets are provably correct. The formal model is also sufficiently constructive to support specification-guided generation of executable maintenance triggers from R2RML mappings.

Lastly, it should be stressed that an LLM is not used to establish the correctness of the changesets. Rather, the LLM-based study provides independent operational evidence that the formal model is sufficiently constructive to be translated into executable maintenance code.

The remainder of this article is organized as follows. Section 2 summarizes related work. Section 3 reviews the basic concepts of RDB2RML views. Section 4 describes a case study. Section 5 introduces the proposed incremental view maintenance formalism. Section 6 discusses the operational architecture. Section 7 describes the experiments conducted and discusses the results. Section 8 concludes the article. Finally, the appendix contains the proofs of the main results.

2 Related Work

Incremental View Maintenance (IVM). The Incremental View Maintenance problem has been studied in the literature for a very long time [25].

Early work covered relational views [10], object-oriented views [2], semi-structured views [39], and XML views [15], just to name a few efforts.

Some more recent approaches include the following. As for generic IVM tools, DBSP [9] introduces an expressive language that enables efficient incremental view maintenance for queries written in multiple languages, including SQL and Datalog. Enzyme [37] is an IVM engine developed at Databricks to power Spark Declarative Pipelines and provides a built-in, end-to-end approach to incremental pipelines that utilizes materialized views as first-class building blocks. OpenIVM [7] is an SQL-to-SQL compiler for Incremental View Maintenance (IVM) that compiles view definitions into SQL to propagate deltas into the table that materializes the view eventually. `pg_ivm`¹ is the most prominent, production-ready implementation of IVM for PostgreSQL.

As for IVM SPARQL and graph databases, Incremunica [34] integrates multiple incremental operators, allowing it to adapt to different queries and data for optimal performance, supports federated querying by dynamically adding and removing sources during query execution, provides SPARQL Query Language support, supports multiple IVM techniques, and enables client-side execution.

¹ https://github.com/sraoss/pg_ivm

The counting algorithm, a classic approach to incremental view maintenance for queries over relational data, is adapted in [31] to SPARQL queries over RDF datasets. Graph property views, both virtual and materialized, are considered in [13] for integrating heterogeneous data using rewriting techniques.

Solutions in specific settings include the following. The maintenance of self-join-free conjunctive queries over a dynamic database, where tuples can be inserted or deleted, is investigated in [14]. A maintenance approach that combines delta queries, trees of materialized views, and heavy-light data partitioning is introduced in [1], whose update time matches or improves the best update time reported in prior work. The incorporation of knowledge about foreign keys into IVM to speed up performance is investigated in [32], which reports experiments in Redshift showing that the proposed technique improved the execution times of the whole refresh process up to 2 times, and up to 2.7 times the process of calculating the necessary changes that will be applied to the materialized view. Finally, a framework for maintaining sketches incrementally under updates is introduced in [17].

The main idea that distinguishes the formal model proposed in [4] from previous work on incremental view maintenance is to explore the entity-preservation property of typical RDB2RDF views, which enables the identification of the specific tuples relevant to a data source update. Only the portion of the view associated with the relevant tuples should be rematerialized. This earlier work was further modified and enhanced to compute changesets for RDB2RDF views [3].

However, such earlier approaches depended on accessing the database state before the update to generate the view changesets. The formal model proposed in this paper addresses this situation by computing the view changesets from the post-update database state, the current view state, and the update itself.

The definition of the formal model proposed in this paper co-evolved with the prompts used in Section 7. An LLM served as an operationalization instrument, and not to establish the correctness of the proposed model, which follows from the formal results. During the development of the formal model, attempts to generate executable maintenance logic from the definitions exposed implicit assumptions and formulations that, although meaningful, were not sufficiently constructive for direct implementation. The resulting feedback led to clearer definitions and a more explicit connection between the formal changeset semantics and the data available to statement-level AFTER triggers.

Formal Software Engineering, Code Verification, and LLMs. In formal software engineering (SE), LLMs have been employed, for example, to address formal requirements engineering [12], generate verifiable specifications from informal requirements [8], help with conceptual modeling [19], facilitate software testing [6,35], and build trustworthy AI agents [38].

The application of LLMs to code verification tasks has also received considerable attention [11], for example, including code-invariant generation [21,22] and reasoning about program semantics [16].

Specification-guided LLM Synthesis. Specification-guided LLM synthesis refers to the process of generating executable artifacts from formally defined semantic specifications.

For example, Murphy et al. [24] divided code generation into two parts: one handled by an LLM and the other by formal methods-based program synthesis. VERGE [30] decomposes LLM outputs into atomic claims, autoformalizes them into first-order logic, and verifies their logical consistency using automated theorem proving. SGCR [36] is a framework that grounds LLMs in human-authored specifications to produce trustworthy and relevant feedback.

Specifically with respect to SQL code generation, the text-to-SQL task, that is, the generation of SQL from Natural Language descriptions, has received considerable attention. Recent LLM-based text-to-SQL tools are surveyed in [18,20,28,29]. The Text-to-SQL Handbook² lists the best-performing text-to-SQL tools on several benchmarks.

This paper takes a different direction and shows that LLMs are capable of processing a (complex and lengthy) formal model and outputting the formally specified database code. Specifically, the paper first shows how to translate R2RML statements into transformation rules using an LLM prompted with the formal model. It then demonstrates how to translate the transformation rules into the required AFTER triggers, again using an LLM prompted with the formal model.

3 RDB2RDF Views

3.1 Basic Concepts and Notation

As usual, a *relation scheme* is denoted as $R[A_1, \dots, A_n]$. The *relational constraints* considered in this article consist of *mandatory* (or not null) *attributes*, *keys*, *primary keys* and *foreign keys*. In particular, $F(R:L, S:K)$ denotes a foreign key, named F , that *relate* R and S , where L and K are lists of attributes from R and S , respectively, with the same length.

A *relational schema* is a pair $\mathbf{S} = (\mathbf{R}, \Omega)$, where $\mathbf{R}$ is a set of relation schemes and Ω is a set of relational constraints such that: (i) Ω has a unique primary key for each relation scheme in $\mathbf{R}$; (ii) Ω has a mandatory attribute constraint for each attribute which is part of a key or primary key; (iii) if Ω has a foreign key of the form $F(R:L, S:K)$, then Ω also has a constraint indicating that K is the primary key of S . The *vocabulary* of $\mathbf{S}$ is the set of relation names, attribute names, and foreign key names used in $\mathbf{S}$. Given a relation scheme $R[A_1, \dots, A_n]$ and a *tuple variable* t over R , $t.A_k$ denotes the *projection* of t over A_k . *Selections* over relation schemes are defined as usual.

A *state* σ of a relational schema $\mathbf{S}$ assigns to each relation scheme R of $\mathbf{S}$ a relation $R(\sigma)$, in the usual way. Let $\mathbf{S} = (\mathbf{R}, \Omega)$ be a relational schema and R and T be relation schemes of $\mathbf{S}$.

² https://github.com/HKUSTDial/NL2SQL_Handbook

An *ontology vocabulary*, or simply a *vocabulary*, is a set of *class names*, *object property names* and *datatype property names*. An *ontology* is a pair $\mathbf{O} = (V, \Sigma)$ such that V is a vocabulary and Σ is a finite set of formulae in V , the *constraints* of $\mathbf{O}$. The constraints include the definition of the *domain* and *range* of a property, as well as *cardinality constraints*, defined in the usual way.

A *foreign key* is an expression of the form $F(R : K, S : L)$, where:

- F is the *name* of the foreign key;
- R and S are relation schemes;
- K and L are lists of attributes of R and S , respectively;
- $|K| = |L|$.

The relation schemes R and S are said to be *related* by F . The foreign key $F(R : K, S : L)$ induces a binary relationship between tuples of R and S and may be traversed in either direction, from R to S or from S to R . Lastly, a tuple $r \in R(\sigma)$ is related (or connected) to a tuple $s \in S(\sigma)$ with respect to F if and only if the values of the attributes in K of r match the values of the attributes in L of s .

A *relational path* φ is a list of the form $[e_1, \dots, e_{n-1}]$, where

- e_j is either a foreign key name F_j , indicating that the path traverses F_j from R_j to R_{j+1} , or an expression of the form $(F_j)^{-1}$, indicating that the path traverses the foreign key F_j from R_{j+1} to R_j , in the reverse direction;
- when e_j is traversed in the forward direction, then the second relation in e_j must be equal to the first relation of e_{j+1} ;
- when e_j is traversed in the reverse direction, then the first relation in e_j must be equal to the first relation of e_{j+1} .

The set of all relation schemes occurring along φ is denoted by $Relations(\varphi)$.

A *prefix* of φ is any sequence $\phi = [e_1, \dots, e_k]$ with $0 \leq k \leq n - 1$.

Given a database state σ , a tuple $r_1 \in R_1(\sigma)$ is *connected to* (or *references*) a tuple $r_n \in R_n(\sigma)$ *by following the path* $\varphi = [e_1, \dots, e_{n-1}]$ if and only if there exists a sequence of tuples $r_2, \dots, r_{n-1}$ with $r_i \in R_i(\sigma)$ for $2 \leq i \leq n - 1$ such that, for each $i \in \{1, \dots, n - 1\}$, the tuples r_i and r_{i+1} are related with respect to the foreign key F_i in the forward or reverse direction.

A relational path φ induces a *predicate path* $\varphi(x, y)$, which is evaluated in a database state by joining the relations in the path in the directions indicated, using the lists of attributes in the foreign key.

An example of a predicate path with two foreign keys is

$$[(place_annotation_fk_place)^{-1}, \\ place_annotation_fk_annotation](p, an)$$

Note that the first expression $(place_annotation_fk_place)^{-1}$ indicates that the foreign key is traversed in the reverse direction.

Table 1. Transformation Rule Patterns

TR	Transformation Rule Pattern
CTR	$\psi : C(s) \leftarrow R(r), URI_R(r, s), \delta[r]$ where - ψ is the name of the transformation rule - R is the name of a relation or a view in the source schema $\mathcal{S}$ - r is a tuple variable bound to tuples of R - s is the URI associated with the tuple bound to r - $URI_R(r, s)$ is a URI constructor predicate as defined in the text - $\delta[r]$ is an optional selection over R , among other relations
Local DTR	$\psi : P(s, v) \leftarrow R_D(r), B_D[r, s], nonNull(r.A_1), \dots, nonNull(r.A_k),$ $RDFLiteral(r.A_1, "A'_1", "G'', v_1), \dots, RDFLiteral(r.A_k, "A'_k", "G'', v_k),$ $T([v_1, \dots, v_k], v), \delta[r]$ where - ψ is the name of the transformation rule - P is a datatype property of V with domain D - $nonNull$ and $RDFLiteral$ are predicates as defined in the text - " $R_D(r), B_D[r, s]$ " is the RHS of the CTR ψ_D that matches class D with pivot relation R_D of ψ_D - $A_1, \dots, A_k$ are attributes of R_D - T is an optional function that transforms values of attributes $A_1, \dots, A_k$ of R_D to values of property P - $\delta[r]$ is an optional selection over R_D , among other relations
Path DTR	$\psi : P(s, v) \leftarrow R_D(r), B_D[r, s], nonNull(r.A_1), \dots, nonNull(r.A_k),$ $RDFLiteral(r.A_1, "A'_1", "G'', v_1), \dots, RDFLiteral(r.A_k, "A'_k", "G'', v_k),$ $T([v_1, \dots, v_k], v), \varphi(r, g), \delta[r, g]$ where - $\varphi(r, g)$ is a predicate path starting on R_D and ending on a relation R_G - $\delta[d, g]$ is an optional selection over R_D and R_G , among other relations - the other terms are as in a Path DTR
Regular OTR	$\psi : P(s, o) \leftarrow R_D(d), B_D[d, s], R_G(g), B_G[g, o], \varphi(d, g), \delta[d, g]$ where - " $R_G(g), B_G[g, o]$ " is the RHS of the CTR ψ_G that matches class G with pivot relation R_G of ψ_G - the other terms are as in a Path DTR
Reification OTR	$\psi : P(s, o) \leftarrow R_D(d), B_D[d, s], nonNull(r.A_1), \dots, nonNull(r.A_k),$ $RDFLiteral(r.A_1, "A'_1", "G'', v_1), \dots, RDFLiteral(r.A_k, "A'_k", "G'', v_k),$ $T([v_1, \dots, v_k], v), RDFIRI(u, o), \delta[r]$ where - $RDFIRI(u, o)$ is as defined in the text - the other terms are as in a Local DTR

3.2 Specification of Entity-Preserving RDB2RDF View

This section presents the formalism for specifying entity-preserving RDB2RDF views. By restricting ourselves to this class of views, we can precisely identify the tuples relevant to a data source update with respect to an RDB2RDF view.

The formal definition of an RDB2RDF view is similar to that given in [26,27]. An *RDB2RDF view* is a triple $\mathcal{W} = (V, \mathcal{S}, M)$, where:

1. V is the vocabulary of the *target ontology*;
2. $\mathcal{S}$ is the *source relational schema*; and
3. M is a set of *mappings* between V and $\mathcal{S}$, defined by *transformation rules* (TRs).

A view $\mathcal{W} = (V, \mathcal{S}, M)$ satisfies the *entity-preserving property* iff:

- the instances of the classes in V correspond to tuples in selected relations of $\mathcal{S}$, which are called the *pivot relations* of $\mathcal{W}$, so that distinct tuples from the same pivot relation generate distinct class instances;
- the values of datatype properties in V of these instances are given by (functions of) attribute values of the corresponding tuples, or of related tuples;
- the object properties in V correspond to relationships between tuples of the pivot relations of $\mathcal{S}$.

Intuitively, a view satisfies the entity-preserving property iff it preserves the base entities (objects) of the source database, rather than creating new entities from the existing ones [23].

The problem of generating transformation rules is addressed in [4,5] and is outside of the scope of this work. However, Table 1 summarizes a set of Transformation Rule (TR) patterns that lead to the definition of relational to RDF mappings that guarantee that the RDB2RDF views satisfy the entity-preserving property by construction.

Table 1 defines three types of TR patterns: *Class Transformation Rule* (CTR), *Datatype Property Transformation Rule* (DTR), and *Object Property Transformation Rule* (OTR).

Intuitively, a CTR ψ maps tuples of a relation R into RDF instances of a class C . It establishes a semantic correspondence between a pivot tuple r and an RDF resource x , such that each pivot tuple is associated with at most one RDF instance, and distinct pivot tuples generate distinct RDF resources. Thus, the mapping preserves the identity of relational entities in the RDB2RDF view.

There are two types of DTRs. A *Local DTR* has no relational path in the body so that the datatype values are obtained directly from attribute values of the pivot tuples. A *Path DTR* has a relational path in the body, and the datatype values are obtained from tuples reachable through the relational path from the pivot tuples.

There are also two types of OTRs. A *regular OTR* Ψ defines object properties by relating an RDF instance generated from a pivot tuple t to an RDF instance generated from a pivot tuple u reached from t through a relational path in the source schema. Hence, OTRs capture relationships between entities represented in the database.

A *reification OTR* relates an RDF instance generated from a pivot tuple t to a URI obtained by reification of a literal l created from attribute values of a pivot tuple u reached from t through a relational path in the source schema.

In addition to predicate paths, the transformation rules in Table 1 and in the experiments use several predicates. The simplest ones are:

- $nonNull(v)$ holds iff value v is not null.
- $RDFLiteral(u, A, R, v)$ holds iff v is the literal representation of a value u of attribute A of R .
- $RDFIRI(u, o)$ holds iff the non-null value u is converted into the RDF IRI o (that is, o is just the literal u treated as a URI).
- $F(r, s)$ holds iff a tuple r of R is related to tuple s of S by a foreign key of the form $F(R:L, S:K)$.

The more complex predicates are as follows.

Let R be the pivot relation of a transformation rule Ψ , and r be a pivot tuple of R .

A *URI template* for R is a string pattern p with place holders $\langle h_1, \dots, h_n \rangle$ such that, when the place holders are appropriately instantiated, the resulting string becomes a URI.

A URI s is associated with r via p iff the predicate

$$hasURI(p, \langle e_1, \dots, e_n \rangle, s)$$

holds, where

- $\langle e_1, \dots, e_n \rangle$ is an ordered sequence of expressions, where each e_i is of the form:
 - $e_i = r.A$, where $r.A$ denotes an attribute of the pivot tuple.
 - $e_i = [FK(r, t)/t.A]$, where $[FK(r, t)/t.A]$ denotes the value of attribute A obtained after following the foreign key FK from the pivot tuple r to tuple t such that the foreign-key navigation must be functional from the pivot relation to the referenced relation.
- s is the result of instantiating each h_i by e_i in p .

To simplify the notation, a second predicate, called a *URI constructor*, is introduced by definition:

$$URI_R(r, s) \equiv hasURI(p, \langle e_1, \dots, e_n \rangle, s)$$

As an example, the first line of Table 1 defines the URIs associated with the pivot relation *artist* using the predicate

$$hasURI("http://musicbrainz.org/artist/\langle h \rangle", \langle r.gid \rangle, s)$$

indicating that s is generated by replacing h by " $r.gid$ " in the template

$$"http://musicbrainz.org/artist/\langle h \rangle"$$

If the transformation rule is generated from an R2RML statement (see Section 6), the R2RML subject-map template specifies the complete URI template, and $\langle e_1, \dots, e_n \rangle$ is the ordered sequence of expressions corresponding to the

placeholders of the R2RML URI template. Furthermore, every placeholder for an `rr:template` is translated into exactly one pivot-rooted attribute expression, preserving the complete URI template, the placeholder order, and the relational origin of each value.

The approach proposed in this article efficiently computes changesets by exploring the entity-preserving property, which allows the precise identification of the tuples in the pivot relations whose corresponding instances may have been affected by an update. The advantages of this approach are: (1) it simplifies the specification of the view, as the formalism is specifically designed to describe the correspondences that define an entity-preserving view; (2) the restrictions on the structure of the queries help ensure the consistency of the specification of the view; (3) the formal expressions that define the queries can be explored to construct the procedures automatically that compute the changesets that maintain the RDB2RDF view; (4) it facilitates the task of providing rigorous proofs for the correctness of the proposed approach.

3.3 Materialization of an RDB2RDF View

The state of an RDB2RDF view is represented as a set of quadruples (or *quads*) of the form (s, p, o, g) . In standard N-Quads, the fourth element g identifies the named graph to which the triple (s, p, o) belongs. In this approach, each transformation rule Ψ is associated with a named graph, and its URI is used as the fourth element. Thus, each quad (s, p, o, g) , where g is the URI of a transformation rule Ψ , indicates that the triple (s, p, o) was generated by Ψ .

This representation allows the RDB2RDF view to be materialized as a set of quads partitioned by transformation rules. Intuitively, each rule Ψ induces a named graph containing exactly the triples generated by Ψ , which provides a simple and operational mechanism for provenance tracking and avoids ambiguity in the presence of duplicate triples generated by different rules.

The state of an RDB2RDF View is defined compositionally in three hierarchical levels.

Definition 1 (Hierarchical RDF State Semantics). *Let $W = (V, S, M)$ be an RDB2RDF view and let σ be a relational database state.*

1. *RDF Contribution produced by a Transformation Rule*

Let $\Psi \in M$ be a transformation rule whose pivot relation is R^ , with tuple variables $r_1, \dots, r_n$ associated with relation schemes $R_1, \dots, R_n$.*

For tuples $p_1 \in R_1(\sigma), \dots, p_n \in R_n(\sigma)$, the application of Ψ under σ , denoted by

$$\Psi[r_1 \leftarrow p_1, \dots, r_n \leftarrow p_n](\sigma)$$

is defined as the set of RDF triples produced by evaluating Ψ with bindings $r_i = p_i$, for $i = 1, \dots, n$, over σ .

2. **RDF Contribution of a Pivot Tuple under a Rule.** For a pivot tuple $t \in R^*(\sigma)$, the RDF contribution of t under Ψ and σ , denoted by $RDF_State[\Psi](t, \sigma)$, is defined as:

$$RDF_State[\Psi](t, \sigma) = \{(s, p, o, g) \mid (s, p, o) \in \Psi[r \leftarrow t](\sigma), g = URI(\Psi)\}$$

3. **RDF State of the View W .**

The RDF state of the view W under σ , denoted by $RDF_State(W, \sigma)$, is defined as:

$$RDF_State(W, \sigma) = \bigcup_{\Psi \in M} \bigcup_{t \in pivot(\Psi)(\sigma)} RDF_State[\Psi](t, \sigma)$$

Intuitively, the RDF state of an RDB2RDF view is obtained by aggregating the RDF contributions generated by all transformation rules over all pivot tuples in the database state.

The RDF state can be naturally interpreted as an RDF dataset, where each quad (s, p, o, g) represents a triple (s, p, o) in the named graph identified by $g = URI(\Psi)$. Named graphs are used to encode rule-level provenance.

4 Case Study: *MusicBrainz_RDF*

MusicBrainz [33] is an open music encyclopedia that collects music metadata. The ***MusicBrainz*** relational database is built on the PostgreSQL relational database engine and contains all of MusicBrainz music metadata. This data includes information about artists, release groups, releases, recordings, works, and labels, along with the many relationships among them. Figure 1(a) depicts a fragment of the ***MusicBrainz*** relational database schema (for more information about the original scheme, see [33]). Each relation has a distinct primary key, whose name ends with “id”, except *gid*, which is an Universally Unique IDentifier (UUID) for use in permanent links and external applications. The relations *Artist*, *Medium*, *Release*, *Recording* and *Track*, in Figure 1(a), represent the main concepts. The relation *ArtistCredit* represents an N:M relationship between *Artist* and *Credit*. The labels of the arcs, such as *fk1*, are the names of the foreign keys.

The case study uses an RDB2RDF view, called ***MusicBrainz_RDF***, which is defined over the relational schema in Figure 1(a). Appendix A provides the Data Definition Language (DDL) for the tables utilized in this case study.

Figure 1(b) depicts a fragment of the ontology used for publishing the ***MusicBrainz_RDF*** view. Appendix B shows the OWL ontology vocabulary used in the case study. It reuses terms from three well-known vocabularies, *FOAF* (Friend of a Friend), *MO* (Music Ontology), and *DC* (Dublin Core).

Table 3 shows a set of transformation rules that partially specify the mapping between the relational schema in Figure 1(a) and the ontology in Figure 1(b), obtained with the help of the tool described in [5]. The complete set of transformation rules for the specification of the ***MusicBrainz_RDF*** view is defined

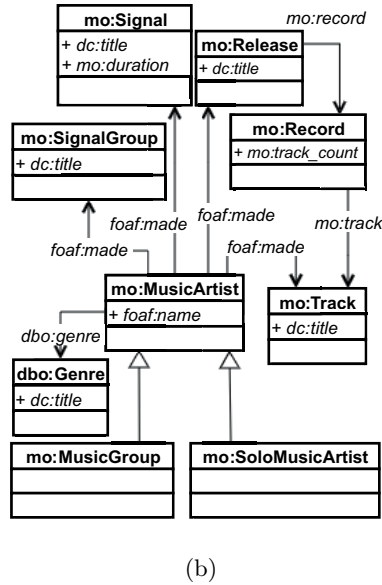
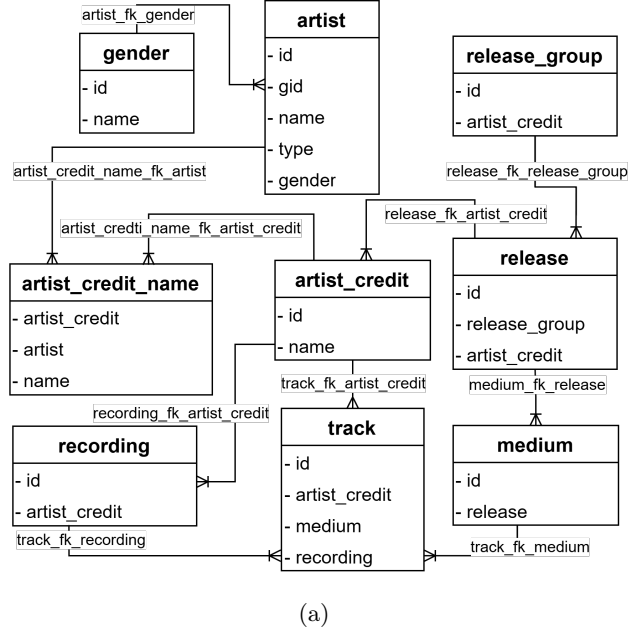


Fig. 1. (a) Fragment of *MusicBrainz* Schema and (b) Fragment of the Music Ontology.

Artist				ArtistCredit			Credit		
aid	gid	name	type	cid	pos	aid	cid	name	
a1	ga1	Kungs	1	c1	1	a1	c1	Kungs vs. Cookin' on 3 B.	
a2	ga2	Cookin' on 3 B.	2	c1	2	a2	c2	Cookin' on 3 B. feat. Kylie Auldist	
a3	ga3	Kylie Auldist	1	c2	1	a2	c3	Kungs	
Track				c2	2	a3	Release		
tid	mid	name	cid	rid	gid	name	cid		
t1	m1	This Girl	c2	r1	gr1	Layers	c3		
t2	m1	Don't You Know	c3						
Recording				RecordingTag		Tag	Medium		
sid	gid	name	cid	length	sid	qid	qid	name	
s1	gs1	This Girl	c1	3:15	s1	q1	q1	pop	
s2	gs2	Don't You Know	c3	3:04	s1	q2	q2	dance	
					s2	q2			
						ReleaseGroup			
						pid	gid	name	cid
						p1	gp1	Layers	c3

Fig. 2. State Example for Relations in Figure 1(a).

in [3] For the examples in the following sections, consider the database state shown in Figure 2.

The transformation rules ψ_1 and ψ_2 in Table 3 are examples of the CTR Pattern. The CTR ψ_1 indicates that, for each tuple r in *Artist*, one should:

1. Compute the URI s such that $hasURI(mbz:, r.gid, s)=true$.
2. Produce the triple (s *rdf:type* *mo:MusicArtist*).

The CTR ψ_2 indicates that, for each tuple r in *Artist*, where $r.type = 1$, one should:

1. Compute the URI s such that $hasURI(mbz:, r.gid, s)=true$.
2. Produce the triple (s *rdf:type* *mo:SoloMusicArtist*).

Note that attribute *gid* is a key for relation *Artist*. Therefore, different tuples in *Artist* generate distinct URIs, i.e., distinct instances. Also, note that ψ_1 and ψ_2 use the same predicate *hasURI*. Therefore, if a tuple r is mapped to triples (x *rdf:type* *mo:MusicArtist*) and (y *rdf:type* *mo:SoloMusicArtist*), then $x=y$.

Considering the database state in Figure 2, CTRs ψ_1 and ψ_2 produce the following triples:

$mbz:r.ga1$ *rdf:type* *mo:MusicArtist*)
 $(mbz:r.ga1$ *rdf:type* *mo:SoloMusicArtist*)
 $(mbz:r.ga2$ *rdf:type* *mo:MusicArtist*)
 $(mbz:r.ga3$ *rdf:type* *mo:MusicArtist*)
 $(mbz:r.ga3$ *rdf:type* *mo:SoloMusicArtist*)

The transformation rule ψ_{10} , in Table 3, is an example of the DTR Pattern. The predicates *nonNull*(v) and *RDFLiteral*(u, A, R, v) are as previously defined.

Table 2. Global URI predicates

Predicate	Global URI
<i>URI_artist</i> (a, x)	$\leftarrow hasURI("http://musicbrainz.org/artist/<h>", <a.gid>, x)$
<i>URI_area</i> (ar,x)	$\leftarrow hasURI("http://musicbrainz.org/area/<h>", <ar.gid>, x)$
<i>URI_label</i> (l,x)	$\leftarrow hasURI("http://musicbrainz.org/label/<h>", <l.gid>, x)$
<i>URI_medium</i> (m,x)	$\leftarrow hasURI("http://musicbrainz.org/record/<h>", <m.id>, x)$
<i>URI_place</i> (p,x)	$\leftarrow hasURI("http://musicbrainz.org/place/<h>", <p.gid>, x)$
<i>URI_recording</i> (r,x)	$\leftarrow hasURI("http://musicbrainz.org/recording/<h>", <r.gid>, x)$
<i>URI_release</i> (r,x)	$\leftarrow hasURI("http://musicbrainz.org/release/<h>", <r.gid>, x)$
<i>URI_signalGroup</i> (rg,x)	$\leftarrow hasURI("http://musicbrainz.org/signal-group/<h>", <rg.gid>, x)$
<i>URI_track</i> (t,x)	$\leftarrow hasURI("http://musicbrainz.org/track/<h>", <t.id>, x)$
<i>URI_tag</i> (tg,x)	$\leftarrow hasURI("http://musicbrainz.org/tag/<h>", <tg.name>, x)$
<i>URI_work</i> (w,x)	$\leftarrow hasURI("http://musicbrainz.org/work/<h>", <w.gid>, x)$
<i>URI_releaseEvent</i> (re,x)	$\leftarrow hasURI("http://musicbrainz.org/release/<h1,h2>",$ $\quad <re.release_gid, re.country_gid>, x)$ $\quad \text{with fragment pattern } \text{'\#country_gid'}$
<i>URI_composition</i> (c,x)	$\leftarrow hasURI("http://musicbrainz.org/work/<h1,h2>",$ $\quad <c.work_gid, "composition">, x)$
<i>URI_*_tagging</i> (x,s)	$\leftarrow hasURI(\text{entity-prefix}, <\text{entity identifier}, \text{tag_name}>, s),$ $\quad \text{matching the R2RML pattern 'entityURI\#tag/name'}$

Table 3. Transformation Rules

TR Transformation Rules	
Ψ_1	$mo:MusicArtist(s) \leftarrow artist(a), URI_artist(a, s)$
Ψ_2	$mo:SoloMusicArtist(s) \leftarrow artist(a), URI_artist(a, s), (a.type = 1)$
Ψ_3	$mo:MusicGroup(s) \leftarrow artist(a), URI_artist(a, s), (a.type = 2)$
Ψ_4	$mo:Record(s) \leftarrow medium(m), URI_medium(m, s)$
Ψ_5	$mo:Track(s) \leftarrow track(t), URI_track(t, s)$
Ψ_6	$mo:Release(s) \leftarrow release(r), URI_release(r, s)$
Ψ_7	$foaf:made(s, o) \leftarrow artist(a), URI_artist(a, s), track(t), URI_track(t, o),$ $[artist_credit_name_fk_artist^{-1},$ $artist_credit_name_fk_artist_credit,$ $track_fk_artist_credit^{-1}](a, t)$
Ψ_8	$mo:track(s, g) \leftarrow medium(m), URI_medium(m, s), track(t), URI_track(t, g)$ $[track_fk_medium^{-1}](m, t)$
Ψ_9	$mo:record(s, o) \leftarrow release(r), URI_release(r, s), medium(m), URI_medium(m, o),$ $[medium_fk_release^{-1}](r, m)$
Ψ_{10}	$foaf:name(s, v) \leftarrow artist(a), URI_artist(a, s), nonNull(a.name),$ $RDFLiteral(a.name, "name", "artist", v)$
Ψ_{11}	$mo:track_count(s, v) \leftarrow medium(m), URI_medium(m, s), nonNull(m.track_count),$ $RDFLiteral(m.track_count, "track_count", "medium", v)$
Ψ_{12}	$dc:title(s, v) \leftarrow track(t), URI_track(t, s), nonNull(t.name),$ $RDFLiteral(t.name, "name", "track", v)$
Ψ_{13}	$dc:title(s, v) \leftarrow release(r), URI_release(r, s), nonNull(r.name),$ $RDFLiteral(r.name, "name", "release", v)$
Ψ_{14}	$mo:Signal(s) \leftarrow recording(r), URI_recording(r, s)$
Ψ_{15}	$mo:SignalGroup(s) \leftarrow release_group(rg), URI_signalGroup(rg, s)$
Ψ_{16}	$dbo:Genre(s) \leftarrow tag(tg), URI_tag(tg, s)$
Ψ_{17}	$dc:title(s, v) \leftarrow recording(r), URI_recording(r, s), nonNull(r.name),$ $RDFLiteral(r.name, "name", "recording", v)$
Ψ_{18}	$mo:duration(s, v) \leftarrow track(t), URI_track(t, s), nonNull(t.length),$ $RDFLiteral(t.length, "length", "track", v)$
Ψ_{19}	$dc:title(s, v) \leftarrow release_group(rg), URI_signalGroup(rg, s), nonNull(rg.name),$ $RDFLiteral(rg.name, "name", "release_group", v)$
Ψ_{20}	$dc:title(s, v) \leftarrow tag(tg), URI_tag(tg, s), nonNull(tg.name),$ $RDFLiteral(tg.name, "name", "tag", v)$
Ψ_{21}	$foaf:made(s, o) \leftarrow artist(a), URI_artist(a, s), release(r), URI_release(r, o)$ $[artist_credit_name_fk_artist^{-1},$ $artist_credit_name_fk_artist_credit,$ $release_fk_artist_credit^{-1}](a, r)$
Ψ_{22}	$foaf:made(s, o) \leftarrow artist(a), URI_artist(a, s), release_group(rg), URI_signalGroup(rg, o)$ $[artist_credit_name_fk_artist^{-1},$ $artist_credit_name_fk_artist_credit,$ $release_group_fk_artist_credit^{-1}](a, rg)$
Ψ_{23}	$foaf:made(s, o) \leftarrow artist(a), URI_artist(a, s), recording(r), URI_recording(r, o)$ $[artist_credit_name_fk_artist^{-1},$ $artist_credit_name_fk_artist_credit,$ $recording_fk_artist_credit^{-1}](a, r)$
Ψ_{24}	$dbo:genre(s, v) \leftarrow artist(a), URI_artist(a, s), gender(g),$ $nonNull(g.name), lower(g.name, u), RDFLiteral(u, "name", "gender", v),$ $[artist_fk_gender](a, g)$

Rule ψ_{10} matches the value of attribute *name* of relation *Artist* with the value of datatype property *foaf:name*, whose domain is *mo:MusicArtist*. It indicates that, for each tuple r of R , one should:

1. Compute the URI s for the instance of *mo:MusicArtist* that r represents, using the CTR ψ_1 . Therefore, s and r are semantically equivalent.
2. For each value v , where v is the literal representation of $r.name$ and $r.name$ is not NULL, produce triple (s *foaf:name* v).

Considering the database state in Figure 2, ψ_{10} produces the following triples:

$(mbz:ga1 \text{ foaf:name } \text{"Kungs"})$
 $(mbz:ga2 \text{ foaf:name } \text{"Cookin's on 3 B."})$
 $(mbz:ga3 \text{ foaf:name } \text{"Kylie Auldist"})$

The transformation rules ψ_7 and ψ_8 , in Table 3, are examples of the OTR Pattern. The predicate $F(r, s)$, where F is a foreign key of the form $F(R:L, S:K)$, is as previously defined. For example, rule ψ_7 matches a relationship between a tuple r of *Artist* and a tuple r_3 in *Track* to instances of the object property *foaf:made*, whose domain is *mo:MusicArtist* and range is *mo:Track*. It indicates that, for each tuple r of R , one should:

1. Compute the URI s for the instance of *mo:MusicArtist* that r represents, using the CTR ψ_1 . Therefore, s and r are semantically equivalent.
2. For each tuple r_3 of *Track* such that r is related to r_3 through path

$$[(artist_credit_name_fk_artist)^{-1}, \\ artist_credit_name_fk_artist_credit, \\ (track_fk_artist_credit)^{-1}](r, t)$$

compute the URI g for the instance of *mo:Track* that r_3 represents (using CTR ψ_5). Therefore, r_3 and g are semantically equivalent.

3. Produce triple (s *foaf:made* g)

Considering the database state in Figure 2, ψ_7 produces the following triples:

$(mbz:ga1 \text{ foaf:made } mbz:t2)$
 $(mbz:ga2 \text{ foaf:made } mbz:t1)$
 $(mbz:ga3 \text{ foaf:made } mbz:t1)$

Example 1. Consider the transformation rules for the **MusicBrainz RDF** view defined in Table 3. Also, consider the relation *Artist* in Figure 1(a), which is the pivot relation of the Transformation Rules (TRs) $\Psi_1, \Psi_2, \Psi_3, \Psi_7, \Psi_{10}$, and Ψ_{24} . Thus, the RDF state of a tuple p in relation *Artist* is computed by applying the TRs $\Psi_1[r/p], \Psi_2[r/p], \Psi_3[r/p], \Psi_7[r/p], \Psi_{10}[r/p]$, and $\Psi_{24}[r/p]$. Considering

the database state in Figure 2, the RDF state of tuple $a1$ in *Artist* contains the following quads:

$(mbz:ga1 \text{ rdf:type } mo:MusicArtist \ \Psi_1)$
 $(mbz:ga1 \text{ rdf:type } mo:SoloMusicArtist \ \Psi_2)$
 $(mbz:ga1 \text{ foaf:made } mbz:t2 \ \Psi_7)$
 $(mbz:ga1 \text{ foaf:name } "Kungs" \ \Psi_{10})$
 $(mbz:ga1 \text{ dbo:genre } mbz:q1 \ \Psi_{24})$
 $(mbz:ga1 \text{ dbo:genre } mbz:q2 \ \Psi_{24})$
 $(mbz:ga1 \text{ dbo:genre } mbz:q2 \ \Psi_{24})$

We use Ψ interchangeably to denote both a transformation rule and its associated URI.

Note that, $\Psi_3[r/a1]$ produces no triple, while $\Psi_{24}[r/a1]$ produces duplicated triples. The triple $(mbz:ga1 \text{ dbo:genre } mbz:q2)$ is generated twice by assigning different tuple variables in Ψ_{24} . Duplicate triples arise when multiple join bindings of the same transformation rule produce identical RDF triples. This typically occurs when different relational paths or join combinations connect the same pivot tuple to equivalent target tuples.

5 Post-Update Incremental Maintenance of RDB2RDF Views

5.1 The Incremental Maintenance Problem

The diagram in Figure 3 describes the problem of computing a correct changeset for a materialized RDB2RDF view $\mathcal{W}$, when an update u occurs in the source relational database $\mathcal{S}$. In the diagram of Figure 3, assume that:

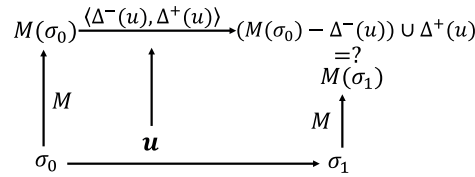


Fig. 3. The problem of computing the correct changeset for RDB2RDF View.

1. M is the set of mappings that materialize view $\mathcal{W}$;
2. σ_0 and σ_1 are the states of $\mathcal{S}$ respectively before and after the update u ; and
3. $M(\sigma_0) = \text{RDF_State}(\mathcal{W}, \sigma_0)$ and $M(\sigma_1) = \text{RDF_State}(\mathcal{W}, \sigma_1)$ are the materializations of $\mathcal{W}$ respectively at σ_0 and σ_1 .

Let u be an update and σ_0 and σ_1 be the database states before and after u , respectively. A *correct changeset* for $\mathcal{W}$ w.r.t. u , σ_0 and σ_1 is a pair $\langle \Delta^-(u), \Delta^+(u) \rangle$, where $\Delta^-(u)$ is a set of triples removed from $\mathcal{W}$ and $\Delta^+(u)$ is a set of triples added to $\mathcal{W}$, that satisfies the following restriction (see Figure 3):

$$M(\sigma_1) = (M(\sigma_0) \setminus \Delta^-(u)) \cup \Delta^+(u) \quad (1)$$

Putting this in words, the changeset $\langle \Delta^-(u), \Delta^+(u) \rangle$ is correctly computed iff the new view state $(M(\sigma_0) \setminus \Delta^-(u)) \cup \Delta^+(u)$, computed with the help of the changeset, and the new view state $M(\sigma_1)$, obtained by rematerializing the view using the view mappings are identical. In this section, we prove that the incremental formulation of changeset computation preserves the semantic effect of the original framework. Specifically, we show that applying the incremental changeset to the pre-update RDF state yields exactly the RDF state of the view after the update.

Database triggers may be defined to compute and publish the correct changeset for the RDB2RDF view, ensuring that the RDF materialization remains synchronized with the underlying relational database [3].

The first step of the strategy is to identify the *relations in the source database that are relevant to the RDB2RDF view*, that is, those relations whose updates may affect the RDF state of the view.

For each update operation u performed on a relevant relation R , a single statement-level **AFTER** trigger is defined. The **AFTER** trigger is fired immediately after the execution of the update and is responsible for computing $\Delta^-(u)$.

Although $\Delta^-(u)$ is formally defined over the pre-update database state σ_0 , its computation is performed inside an **AFTER** trigger, where the database is already in the post-update state σ_1 . Thus, even though the trigger executes after the update, the combination of the deleted and inserted tuple sets, along with the current database state and the old view state, allows for the correct computation of the changesets. Consequently, $\Delta^-(u)$ is computed precisely as defined over the pre-update database state.

5.2 A Post-Update Formulation of Changeset Computation

Let $u = (D, I)$ be an update over a relation R , let σ_1 be the database state after applying u , and let $\mathcal{W} = (V, \mathcal{S}, M)$ be an RDB2RDF view. In most definitions, an explicit mention of σ_1 will be omitted to simplify the notation slightly.

Definition 2 (Transformation Rules Relevant to a Relation). *Let Ψ be a transformation rule and R be a relation.*

1. Ψ is *pivot-relevant to updates on R* iff $\text{pivot}(\Psi) = R$, that is, R is the first relation in the body of Ψ .
2. Ψ is *relation-relevant to updates on R* iff R occurs in the body of Ψ in an occurrence other than the pivot occurrence.
3. Ψ is *relevant to updates on R* iff Ψ is pivot-relevant or relation-relevant to updates on R ; $\text{Relev}(R)$ denotes the set of transformation rules relevant to updates on R .

If Ψ is *pivot-relevant* to updates on R , the tuples in D and I are themselves pivot tuples of Ψ , and their RDF contributions are directly affected by the update. If Ψ is *relation-relevant* to updates on R , the update affects the evaluation of Ψ indirectly: tuples in the pivot relation R^* of Ψ whose derivations depend on tuples in D or I may have their RDF contributions modified. Note that the two conditions are not mutually exclusive: Ψ may be both pivot-relevant and relation-relevant to R .

The changesets $\Delta^-(u)$ and $\Delta^+(u)$ are computed by aggregating the deletion and insertion contributions of all transformation rules that are relevant to the update. Intuitively, each relevant rule $\Psi \in M$ contributes with a pair

$$< \Delta^-[\Psi](u), \Delta^+[\Psi](u) >$$

where

- $\Delta^-[\Psi](u)$: the incremental deletion contribution is the set of RDF quads that become invalid due to the deletion of tuples in D
- $\Delta^+[\Psi](u)$: the incremental insertion contribution is the set of RDF quads that become valid due to the insertion of tuples in I

The overall changeset is then defined as:

$$\begin{aligned} \Delta^-(u) &= \bigcup_{\Psi \in \text{Relev}(R)} \Delta^-[\Psi](u) \\ \Delta^+(u) &= \bigcup_{\Psi \in \text{Relev}(R)} \Delta^+[\Psi](u) \end{aligned}$$

In the following subsections, we detail how to compute the incremental deletion and insertion contributions $\Delta^-[\Psi](u)$ and $\Delta^+[\Psi](u)$.

5.2.1 Computing the Incremental Deletion Contribution of Relevant Rules

For a transformation rule $\Psi \in M$ that is relevant to an update u over relation R , the incremental deletion contribution can be decomposed into two components:

$$\Delta^-[\Psi](u) = \Delta_{pivot}^-[\Psi](u) \cup \Delta_{rel}^-[\Psi](u)$$

where

- $\Delta_{pivot}^-[\Psi](u)$: denotes the deletion contribution of Ψ arising from its pivot-relevant effect.
- $\Delta_{rel}^-[\Psi](u)$: denotes the deletion contribution of Ψ arising from its relation-relevant effect.

In the following, we first describe how to compute $\Delta_{pivot}^-[\Psi](u)$ and then $\Delta_{rel}^-[\Psi](u)$.

5.2.1.1 Incremental Deletion Contribution of Pivot-Relevant Rules

Let Ψ be a transformation rule and $u = (D, I)$ be an update on R in state σ_0 . Consider that R is the pivot relation of ψ (that is, R is the first occurrence of a relation scheme in $path(\Psi)$), in which case ψ is pivot-relevant to R .

Let $d \in D$ be a deleted pivot tuple. In this case, each RDF triple (s, p, o) generated from d by Ψ in σ_0 becomes invalid, and the quad (s, p, o, g) must be removed from the view, where $g = URI(\Psi)$ encodes the provenance of triples generated by rule Ψ .

In detail, *the incremental contribution of Ψ with respect to a deleted tuple $d \in D$ from relation R* , when Ψ is pivot-relevant to R , is defined as:

$$\Delta_{pivot}^-[\Psi](d) = \{(s, p, o, g) \mid (s, p, o, g) \in W_0, s = URI(d), g = URI(\Psi)\}$$

where W_0 is the view state before the update, $URI(d)$ denotes the RDF subject associated with the pivot tuple d , and $g = URI(\Psi)$.

Then, *the overall incremental contribution of a rule Ψ with respect to all deleted tuples in an update $u = (D, I)$ on R* , when Ψ is pivot-relevant to R , is defined by aggregating the contributions of all deleted pivot tuples:

$$\Delta_{pivot}^-[\Psi](u) = \bigcup_{d \in D} \Delta_{pivot}^-[\Psi](d)$$

Intuitively, since the pivot tuple itself is deleted, all RDF triples previously generated from that tuple become invalid. Under the assumption of entity-preserving mappings, each pivot tuple generates a disjoint set of triples, as distinct pivot tuples are mapped to distinct RDF subjects. Therefore, the deletion contribution can be computed independently for each $d \in D$, and the overall result is simply the union of these per-tuple contributions. (See also Lemma 1 in Section 5.4.)

5.2.1.2 Incremental Deletion Contribution of Relation-Relevant Rules

Let Ψ be a transformation rule and $u = (D, I)$ be an update on R in state σ_0 . Consider an occurrence of R which is not the pivot relation of ψ (that is, the occurrence of R is not the first occurrence of a relation scheme in $path(\Psi)$), in which case ψ is relation-relevant to R . Let R^* be the pivot relation of Ψ .

Let $d \in D$ be a deleted tuple. Then, the update indirectly affects the evaluation of Ψ . Therefore, the computation of the incremental deletion contribution proceeds in two steps: (i) identifying the set of pivot tuples indirectly affected by the deleted tuples D ; and (ii) computing the deletion contribution associated with those affected pivot tuples.

Let $p^* \in R^*(\sigma_0)$. The pivot tuple p^* is said to be *indirectly affected by $u = (D, I)$* iff its RDF contribution depends on at least one tuple in D through the evaluation of Ψ in σ_0 , formally defined as:

$$A^-[\Psi](u) = \{p^* \in R^*(\sigma_0) \mid \exists d \in D, \exists r_k \in VarR(\Psi), \\ \exists (s, p, o) \in \Psi[r^* \leftarrow p^*, r_k \leftarrow d](\sigma_0)\}$$

where $VarR(\Psi)$ is the set of tuple variables in Ψ associated with relation R .

Intuitively, $A[\Psi](u)$ contains all pivot tuples $p^* \in R^*(\sigma_0)$ such that at least one deleted tuple $d \in D$, when bound to some variable r_k of rule Ψ associated with relation R , contributes to the generation of at least one RDF triple by Ψ in the pre-update database state σ_0 .

This definition refers to the pre-update database state σ_0 , but it can be rephrased in terms of the post-update database state σ_1 as follows.

Let $\sigma[R \leftarrow S]$ denote the database state σ' which is equal to σ , except that $R(\sigma') = S$. Then, $A^-[\Psi](u)$, with $u = (D, I)$, can be rephrased as:

$$A^-[\Psi](u) = \{p^* \in R^*(\sigma_1) \mid \exists d \in D, \exists r_k \in VarR(\Psi), \\ \exists (s, p, o) \in \Psi[r^* \leftarrow p^*, r_k \leftarrow d](\sigma_1[R \leftarrow ((R \cup D) \setminus I)])\}$$

Note that $\Psi[r^* \leftarrow p^*, r_k \leftarrow d](\sigma_1[R \leftarrow ((R \cup D) \setminus I)])$ simply says that

- the tuple variable r^* is instantiated with the pivot tuple p^* .
- the tuple variable r_k , associated with some occurrence of R in $path(\Psi)$, is instantiated with some deleted tuple d .
- all other tuple variables associated with other occurrences of R in $path(\Psi)$ are instantiated with tuples from $((R(\sigma_1) \cup D) \setminus I)$, that is, with tuples from $R(\sigma_0)$, since $((R(\sigma_1) \cup D) \setminus I) = R(\sigma_0)$.

In fact, one can prove that the two definitions are equivalent, using two simple facts:

- $R^*(\sigma_0) = R^*(\sigma_1)$, since by assumption $R^* \neq R$ and u is an update on R .
- $\sigma_1[R \leftarrow ((R \cup D) \setminus I)] = \sigma_0$, since $u = (D, I)$ is an update on R and, hence, it follows that $(R(\sigma_1) \cup D \setminus I) = R(\sigma_0)$, and $T(\sigma_1) = T(\sigma_0)$, for any other relation scheme $T \neq R$.

There is also a special case worth considering when R occurs only once in $path(\Psi)$, in which case $A^-[\Psi](u)$ can be equivalently rewritten as:

$$A^-[\Psi](u) = \{p^* \in R^*(\sigma_1) \mid \exists d \in D, \\ \exists (s, p, o) \in \Psi[r^* \leftarrow p^*, r_R \leftarrow d](\sigma_1)\}$$

where r_R is the tuple variable associated with the only occurrence of R in $path(\Psi)$.

This follows because there are no other tuple variables associated with other occurrences of R in $path(\Psi)$ to be instantiated. Hence,

$$\Psi[r^* \leftarrow p^*, r_k \leftarrow d](\sigma_1) = \Psi[r^* \leftarrow p^*, r_k \leftarrow d](\sigma_0)$$

Now, let $p^* \in A^-[\Psi](u)$ and W_0 be the view state before the update. Consider the sets:

$$S_1[\Psi](p^*) = \{(s, p, o, g) \mid (s, p, o, g) \in W_0, s = URI(p^*), g = URI(\Psi)\}$$

$$S_2[\Psi](p^*) = \{(s, p, o, g) \mid (s, p, o) \in \Psi[r^* \leftarrow p^*](\sigma_1), g = URI(\Psi)\}$$

where r^* is the tuple variable associated with the pivot relation R^* .

The set $S_1[\Psi](p^*)$ contains all quads generated by Ψ from p^* in σ_0 (since $(s, p, o) \in W_0$, $s = URI(p^*)$ and $g = URI(\Psi)$), whereas $S_2[\Psi](p^*)$ contains all quads generated by Ψ from p^* in the post-update database state σ_1 . Note that $S_1[\Psi](p^*)$ may include a quad q that is not affected by the update, which is precisely one of the quads in $S_2[\Psi](p^*)$. Therefore, q is preserved by the set difference operation.

The *incremental deletion contribution of Ψ with respect to $p^* \in A^-[\Psi](u)$* , with $u = (D, I)$, when Ψ is relation-relevant to R , is defined as:

$$\Delta_{rel}^-[\Psi](p^*) = (S_1[\Psi](p^*) \setminus S_2[\Psi](p^*))$$

The *overall incremental deletion contribution of a rule $\Psi \in \mathcal{M}$ for $u = (D, I)$* , when Ψ is relation-relevant to R , is defined by aggregating the contributions of all affected pivot tuples:

$$\Delta_{rel}^-[\Psi](u) = \bigcup_{p^* \in A^-[\Psi](u)} \Delta_{rel}^-[\Psi](p^*)$$

5.2.2 Computing the Incremental Insertion Contribution of Relevant Rules

For a transformation rule $\Psi \in \mathcal{M}$ that is relevant to an update u over relation R , the incremental insertion contribution can be decomposed into two components:

$$\Delta^+[\Psi](u) = \Delta_{pivot}^+[\Psi](u) \cup \Delta_{rel}^+[\Psi](u)$$

where

- $\Delta_{pivot}^+[\Psi](u)$: denotes the insertion contribution of Ψ arising from its pivot-relevant effect.
- $\Delta_{rel}^+[\Psi](u)$: denotes the insertion contribution of Ψ arising from its relation-relevant effect

In the following, we first describe how to compute $\Delta_{pivot}^+[\Psi](u)$ and then $\Delta_{rel}^+[\Psi](u)$.

5.2.2.1 Incremental Insertion Contribution of Pivot-Relevant Rules

Let Ψ be a transformation rule and $u = (D, I)$ be an update on R in state σ_0 . Consider that R is the pivot relation of ψ (that is, R is the first occurrence of a relation scheme in $path(\Psi)$), in which case ψ is pivot-relevant to R .

The *incremental contribution of Ψ with respect to an inserted tuple $i \in I$ into relation R* , when Ψ is pivot-relevant to R , is defined as:

$$\Delta_{pivot}^+[\Psi](i) = \{(s, p, o, g) \mid (s, p, o) \in \Psi[r \leftarrow i](\sigma_1), g = URI(\Psi)\}$$

where $\Psi[r \leftarrow i](\sigma_1)$ denotes the set of triples generated by applying rule Ψ to the pivot tuple i , evaluated over the post-update database state σ_1 .

Then, the overall incremental contribution of a rule Ψ with respect to all inserted tuples in an update $u = (D, I)$ on R , when Ψ is pivot-relevant to R , is defined by aggregating the contributions of all inserted pivot tuples:

$$\Delta_{pivot}^+[\Psi](u) = \bigcup_{i \in I} \Delta_{pivot}^+[\Psi](i)$$

5.2.2.2 Incremental Insertion Contribution of Relation-Relevant Rules

Let Ψ be a transformation rule and $u = (D, I)$ be an update on R in state σ_0 . Consider an occurrence of R which is not the pivot relation of ψ (that is, the occurrence of R is not the first occurrence of a relation scheme in $path(\Psi)$), in which case ψ is relation-relevant to R . Let R^* be the pivot relation of Ψ .

In this case, the update affects the evaluation of Ψ indirectly. Therefore, the computation of the incremental insertion contribution proceeds in two steps: (i) identifying the set of pivot tuples indirectly affected by the inserted tuples I ; and (ii) computing the insertion contribution associated with those affected pivot tuples.

A pivot tuple is said to be *indirectly affected* if it is not itself inserted, but its RDF contribution depends on one or more tuples in I through the evaluation of Ψ , formally defined as:

$$A^+[\Psi](u) = \{p^* \in R^*(\sigma_1) \mid \exists i \in I, \exists r_k \in VarR(\Psi), \\ \exists (s, p, o) \in \Psi[r^* \leftarrow p^*, r_k \leftarrow i](\sigma_1)\}$$

where:

- $R^* = \text{pivot}(\Psi)$
- r^* is the tuple variable associated with R^*
- $VarR(\Psi)$ denotes the set of tuple variables in Ψ associated with relation R

There is also a special case worth considering when R occurs only once in $path(\Psi)$, in which case $A^+[\Psi](u)$ can be equivalently rewritten as:

$$A^+[\Psi](u) = \{p^* \in R^*(\sigma_1) \mid \exists i \in I, \\ \exists (s, p, o) \in \Psi[r^* \leftarrow p^*, r_R \leftarrow i](\sigma_1)\}$$

where r_R is the tuple variable associated with the only occurrence of R in $path(\Psi)$.

Then, the *incremental insertion contribution of Ψ with respect to $p^* \in A^+[\Psi](u)$* is defined as:

$$\Delta_{rel}^+[\Psi](p^*) = \{(s, p, o, g) \mid (s, p, o) \in \Psi[r^* \leftarrow p^*](\sigma_1), g = URI(\Psi)\}$$

The set $\Delta_{rel}^+[\Psi](p^*)$ contains all quads generated by Ψ from p^* in the post-update database state σ_1 . Note that duplicate quads may arise; however, this is harmless under the set semantics of RDF. As a result, only the quads that become valid due to the insertion of tuples in I are inserted.

Finally, *the overall incremental insertion contribution of a rule $\Psi \in \mathcal{M}$ for u* , when Ψ is relation-relevant to R , is defined by aggregating the contributions of all affected pivot tuples:

$$\Delta_{rel}^+[\Psi](u) = \bigcup_{p^* \in A^+[\Psi](u)} \Delta_{rel}^+[\Psi](p^*)$$

5.3 An Abstract Algorithm Synthesized from the Formal Definitions

This section presents an abstract algorithm (Algorithm 1) to compute the insertion and deletion changesets $\Delta^-(u)$ and $\Delta^+(u)$. The algorithm basically rewrites the formal definitions in pseudo-code, following a compositional structure, and ensuring that all affected RDF quads are computed using the post-update state σ_1 and the pre-update RDB2RDF view state W_0 .

Importantly, the algorithm was automatically synthesized from the formal definitions using an LLM. This illustrates a key property of the proposed framework, namely *LLM compilability*: the ability of a formal specification to be directly translated into an operational algorithm by an LLM. This suggests a new methodological direction in data management, in which formal theories are designed not only for correctness but also for machine-driven compilation into executable systems.

The algorithm presented in this section is intentionally abstract and should not be interpreted as concrete implementations for generating database triggers. Rather, it provides a procedural interpretation of the formal definitions, serving as a bridge between the declarative specification of the framework and its operational realization. In this sense, the algorithm can be understood as a form of *meta-level code*.

In our approach, LLMs play two complementary roles. First, they are used to synthesize abstract algorithms from the formal definitions. This process serves as a form of validation, ensuring that the definitions are sufficiently precise, complete, and internally consistent to translate into an operational procedure without requiring additional implicit assumptions.

Second, given a concrete specification of the mappings and the target environment, an LLM can be used to generate executable trigger implementations directly from the same formal definitions. This second role is discussed in the following section.

Algorithm 1: COMPUTE_DELTA_R(W, u, R, σ_1, W_0)

Input: RDB2RDF view $W = (V, S, M)$; update $u = (D, I)$ over relation R ;
post-update database state σ_1 ; pre-update RDF state W_0 .

Output: Changeset $\langle \Delta^-(u), \Delta^+(u) \rangle$.

1. $\Delta^-(u), \Delta^+(u) \leftarrow \emptyset$;
2. $Relev(R) \leftarrow \{\Psi \in M \mid \Psi \text{ is pivot-relevant to } R \vee \Psi \text{ is relation-relevant to } R\}$;
3. **foreach** $\Psi \in Relev(R)$ **do**
4. $\Delta_{pivot}^-(\Psi)(u), \Delta_{rel}^-(\Psi)(u) \leftarrow \emptyset$;
5. $\Delta_{pivot}^+(\Psi)(u), \Delta_{rel}^+(\Psi)(u) \leftarrow \emptyset$;
6. // 1. Pivot-relevant contribution
7. **if** Ψ is pivot-relevant to updates on R **then**
8. **foreach** $d \in D$ **do**
9. $\Delta_{pivot}^-(\Psi)(d) \leftarrow \{(s, p, o, g) \in W_0 \mid s = URI(d) \wedge g = URI(\Psi)\}$;
10. $\Delta_{pivot}^-(\Psi)(u) \leftarrow \Delta_{pivot}^-(\Psi)(u) \cup \Delta_{pivot}^-(\Psi)(d)$;
11. **end**
12. **foreach** $i \in I$ **do**
13. $\Delta_{pivot}^+(\Psi)(i) \leftarrow \{(s, p, o, g) \mid (s, p, o) \in \Psi[r \leftarrow i](\sigma_1) \wedge g = URI(\Psi)\}$;
14. $\Delta_{pivot}^+(\Psi)(u) \leftarrow \Delta_{pivot}^+(\Psi)(u) \cup \Delta_{pivot}^+(\Psi)(i)$;
15. **end**
16. // 2. Relation-relevant contribution
17. **if** Ψ is relation-relevant to updates on R **then**
18. $R^* \leftarrow pivot(\Psi)$;
19. // 2.1 Affected pivot tuples for deletion
20. **if** R occurs only once in $path(\Psi)$ **then**
21. $A_{\Psi}^- \leftarrow \{p^* \in R^*(\sigma_1) \mid \exists d \in D, \exists (s, p, o) \in \Psi[r^* \leftarrow p^*, r_R \leftarrow d](\sigma_1)\}$;
22. **end**
23. **else**
24. $\sigma_R^0 \leftarrow (R(\sigma_1) \cup D) \setminus I$;
25. $A_{\Psi}^- \leftarrow \{p^* \in R^*(\sigma_1) \mid \exists d \in D, \exists r_k \in Var_R(\Psi), \exists (s, p, o) \in \Psi[r^* \leftarrow p^*, r_k \leftarrow d](\sigma_1[R \leftarrow \sigma_R^0])\}$;
26. **end**
27. **foreach** $p^* \in A_{\Psi}^-$ **do**
28. $S_1[\Psi](p^*) \leftarrow \{(s, p, o, g) \in W_0 \mid s = URI(p^*) \wedge g = URI(\Psi)\}$;
29. $S_2[\Psi](p^*) \leftarrow \{(s, p, o, g) \mid (s, p, o) \in \Psi[r^* \leftarrow p^*](\sigma_1) \wedge g = URI(\Psi)\}$;
30. $\Delta_{rel}^-(\Psi)(p^*) \leftarrow S_1[\Psi](p^*) \setminus S_2[\Psi](p^*)$;
31. $\Delta_{rel}^-(\Psi)(u) \leftarrow \Delta_{rel}^-(\Psi)(u) \cup \Delta_{rel}^-(\Psi)(p^*)$;
32. **end**
33. // 2.2 Affected pivot tuples for insertion
34. $A_{\Psi}^+ \leftarrow \{p^* \in R^*(\sigma_1) \mid \exists i \in I, \exists r_k \in Var_R(\Psi), \exists (s, p, o) \in \Psi[r^* \leftarrow p^*, r_k \leftarrow i](\sigma_1)\}$;
35. **foreach** $p^* \in A_{\Psi}^+$ **do**
36. $\Delta_{rel}^+(\Psi)(p^*) \leftarrow \{(s, p, o, g) \mid (s, p, o) \in \Psi[r^* \leftarrow p^*](\sigma_1) \wedge g = URI(\Psi)\}$;
37. $\Delta_{rel}^+(\Psi)(u) \leftarrow \Delta_{rel}^+(\Psi)(u) \cup \Delta_{rel}^+(\Psi)(p^*)$;
38. **end**
39. **end**
40. // 3. Overall deletion and insertion contributions
41. $\Delta^-(\Psi)(u) \leftarrow \Delta_{pivot}^-(\Psi)(u) \cup \Delta_{rel}^-(\Psi)(u)$;
42. $\Delta^+(\Psi)(u) \leftarrow \Delta_{pivot}^+(\Psi)(u) \cup \Delta_{rel}^+(\Psi)(u)$;
43. $\Delta^-(u) \leftarrow \Delta^-(u) \cup \Delta^-(\Psi)(u)$;
44. $\Delta^+(u) \leftarrow \Delta^+(u) \cup \Delta^+(\Psi)(u)$;
45. **end**
46. **return** $\langle \Delta^-(u), \Delta^+(u) \rangle$;

5.4 Correctness and Minimality of the Incremental Formulation

This section presents a series of results showing that the incremental formulation of changeset computation preserves the semantic effect of the original framework. Specifically, the results show that applying the incremental changeset to the pre-update RDF state yields exactly the RDF state of the view after the update. The section also proves that the changesets satisfy a minimality criterion. Appendix A contains the proofs.

In what follows, let $\mathcal{W} = (V, \mathbf{S}, M)$ be an RDB2RDF view, $u = (D, I)$ be an update on a relation R , σ_0 and σ_1 be the database states before and after applying u , and $W_1 = \text{RDF_State}(\mathcal{W}, \sigma_1)$ and $W_0 = \text{RDF_State}(\mathcal{W}, \sigma_0)$ be the view states before and after applying u . Let $\Delta^-(u)$ and $\Delta^+(u)$ be the deletion and insertion components defined as in Section 5.

Let σ be a relational database state. Recall that $\mathcal{P}(\mathcal{W})$ denotes the set of pivot relations of $\mathcal{W}$. By Definition 3, the RDF state of $\mathcal{W}$ under σ is given by:

$$\text{RDF_State}(\mathcal{W}, \sigma) = \bigcup_{R^* \in \mathcal{P}(\mathcal{W})} \bigcup_{p^* \in R^*(\sigma)} \text{RDF_State}[R^*](p^*, \sigma)$$

The first lemma shows that each quad has a unique derivation associated with a single pivot tuple and transformation rule.

Lemma 1 (Uniqueness of Quad Generation).

For every quad $(s, p, o, g) \in \text{RDF_State}(\mathcal{W}, \sigma)$, there exists a unique transformation rule $\Psi \in M$ and a unique pivot tuple $p^ \in \text{pivot}(\Psi)(\sigma)$ such that*

$$(s, p, o, g) \in \text{RDF_State}[\Psi](p^*, \sigma)$$

The next two lemmas show that the incremental deletion and the incremental insertion components capture exactly the quads whose derivations are invalidated by deletions, or created by insertions.

Recall that a quad q has a derivation under state σ iff there is a rule Φ , with pivot relation T^* , and a pivot tuple $t^* \in T^*(\sigma)$ such that $q \in \Phi[r \leftarrow t^*](\sigma)$.

Lemma 2 (Characterization of the Deletion Component).

- (a) *(For any quad q) $((q \in W_0 \wedge q \notin \Delta^-(u)) \Rightarrow q \in W_1)$.*
- (b) *(For any quad q) $(q \in W_1 \Rightarrow (q \notin \Delta^-(u) \vee q \in \Delta^+(u)))$.*

Lemma 3 (Characterization of the Insertion Component).

$\Delta^+(u)$ is the set of quads whose derivation becomes valid in σ_1 due to some tuple $i \in I$.

Theorem 1 states that the insertion and deletion changesets are correct.

Theorem 1 (Correctness of the Incremental Changesets).

$$W_1 = (W_0 \setminus \Delta^-(u)) \cup \Delta^+(u)$$

Finally, Theorem 1 has a simple corollary. Let $u = (D, I)$ be an update. Recall that σ_0 and σ_1 be the database states before and after applying u , and $W_0 = \text{RDF_State}(\mathcal{W}, \sigma_0)$ and $W_1 = \text{RDF_State}(\mathcal{W}, \sigma_1)$ are the view states before and after applying u . Let σ_D be the database state after applying the deletions in D , but before applying the insertions in I . Let $W_D = \text{RDF_State}(\mathcal{W}, \sigma_D)$ be the view state corresponding to W_D .

Corollary 1 (Minimality of the Incremental Changesets).

- (a) $\Delta^-(u) = (W_0 \setminus W_D)$
- (b) $(\Delta^+(u) \setminus W_D) = (W_1 \setminus W_D)$
- (c) If $(\Delta^+(u) \cap W_D) = \emptyset$, then $\Delta^+(u) = (W_1 \setminus W_D)$

Corollary 1 suggests that the changesets are minimal in the following sense. Part (a) says that $\Delta^-(u)$ is exactly the set of quads that are removed from the initial view state W_0 as a result of the deletions in the update u (but before the insertions). Likewise, Part (b) states that $\Delta^+(u)$ is the set of quads that are inserted into the intermediate view state W_D , created after the deletions, as a result of the insertions in the update u . However, we have to ignore quads that are already in W_D , since we do not rule out stale insertions that lead to duplicates. Part (c) is a simple consequence of Part (b) and says that, if $\Delta^+(u)$ has no duplicates w.r.t. W_D , then $\Delta^+(u)$ is the minimum set of quads that must be inserted into W_D to create W_1 .

5.5 Example of Computing $\Delta^-(u)$ and $\Delta^+(u)$

We illustrate the computation of the changeset $\langle \Delta^-(u), \Delta^+(u) \rangle$ for an update on relation **Track**, following the formulation of the previous sections. In accordance with the maintenance strategy adopted in the article, we first compute $\Delta^-(u)$ and then $\Delta^+(u)$.

Update. Consider the update

```
UPDATE Track SET cid = 'c1' WHERE mid = 'm1';
```

From the database state in Figure 2, this update affects two tuples of **Track**. Hence, $u = (D, I)$, with $D = t_1, t_2$ and $I = t'_1, t'_2$, where

$$\begin{aligned} t_1 &= (tid = t1, mid = m1, name = "This Girl", cid = c2), \\ t_2 &= (tid = t2, mid = m1, name = "Don't You Know", cid = c3) \\ t'_1 &= (tid = t1, mid = m1, name = "This Girl", cid = c1), \\ t'_2 &= (tid = t2, mid = m1, name = "Don't You Know", cid = c1) \end{aligned}$$

Relevant Rules. The transformation rules relevant to **Track** are:

$$Relev(\mathbf{Track}) = \{\Psi_5, \Psi_7, \Psi_8, \Psi_{12}\}.$$

Among them, Ψ_5 and Ψ_{12} are pivot-relevant, whereas Ψ_7 and Ψ_8 are relation-relevant.

Phase 1: Computing $\Delta^-(u)$. By Section 6.2,

$$\Delta^-(u) = \bigcup_{\Psi \in Relev(\mathbf{Track})} \Delta^-[\Psi](u), \quad \Delta^-[\Psi](u) = \Delta_{pivot}^-[\Psi](u) \cup \Delta_{rel}^-[\Psi](u).$$

Pivot-relevant rules. For Ψ_5 and Ψ_{12} , the deleted tuples in D are themselves pivot tuples. Thus:

$$\begin{aligned} \Delta^-[\Psi_5](u) = \{ & (mbz : t1, rdf : type, mo : Track, \Psi_5), \\ & (mbz : t2, rdf : type, mo : Track, \Psi_5) \} \end{aligned}$$

and

$$\begin{aligned} \Delta^-[\Psi_{12}](u) = \{ & (mbz : t1, dc : title, "This Girl", \Psi_{12}), \\ & (mbz : t2, dc : title, "Don't You Know", \Psi_{12}) \} \end{aligned}$$

Relation-relevant rule Ψ_7 . Rule Ψ_7 is:

$$\begin{aligned} \Psi_7 : foaf:made(s, g) \leftarrow & Artist(r), hasURI(mbz:, r.gid, s), fk1(r, r_1), \\ & fk2(r_1, r_2), fk3(r_2, r_3), Track(r_3), hasURI(mbz:, r_3.tid, g) \end{aligned}$$

Its pivot relation is **Artist**. Since **Track** occurs only once in $path(\Psi_7)$, the affected pivot tuples for the deletion phase are:

$$A^-[\Psi_7](u) = \{p^* \in Artist(\sigma_1) \mid \exists d \in D, \exists (s, p, o) \in \Psi_7[r \leftarrow p^*, r_3 \leftarrow d](\sigma_1)\}$$

Evaluating Ψ_7 with the deleted tuples yields:

$$t_1 \text{ with } cid = c2 \Rightarrow \text{artists } a2, a3, \quad t_2 \text{ with } cid = c3 \Rightarrow \text{artist } a1$$

Hence,

$$A^-[\Psi_7](u) = \{a1, a2, a3\}$$

Now, for each $p^* \in A^-[\Psi_7](u)$, let

$$S_1[\Psi_7](p^*) = \{(s, p, o, g) \mid (s, p, o, g) \in W_0, s = URI(p^*), g = URI(\Psi_7)\}$$

and

$$S_2[\Psi_7](p^*) = \{(s, p, o, g) \mid (s, p, o) \in \Psi_7[r \leftarrow p^*](\sigma_1), g = URI(\Psi_7)\}$$

Then:

$$\Delta_{rel}^{-}[\Psi_7](u) = \bigcup_{p^* \in A^{-}[\Psi_7](u)} (S_1[\Psi_7](p^*) \setminus S_2[\Psi_7](p^*))$$

Before the update, Ψ_7 generates:

$$\begin{aligned} & (mbz : ga1, foaf:made, mbz : t2, \Psi_7) \\ & (mbz : ga2, foaf:made, mbz : t1, \Psi_7) \\ & (mbz : ga3, foaf:made, mbz : t1, \Psi_7) \end{aligned}$$

After the update, Ψ_7 generates:

$$\begin{aligned} & (mbz : ga1, foaf:made, mbz : t1, \Psi_7) \\ & (mbz : ga1, foaf:made, mbz : t2, \Psi_7) \\ & (mbz : ga2, foaf:made, mbz : t1, \Psi_7) \\ & (mbz : ga2, foaf:made, mbz : t2, \Psi_7) \end{aligned}$$

Therefore,

$$\Delta^{-}[\Psi_7](u) = \{(mbz : ga3, foaf:made, mbz : t1, \Psi_7)\}$$

Relation-relevant rule Ψ_8 . Rule Ψ_8 is:

$$\begin{aligned} \Psi_8 : mo:track(s, g) \leftarrow Medium(r), hasURI(mbz:, r.mid, s), fk4(r, f), \\ Track(f), hasURI(mbz:, f.tid, g) \end{aligned}$$

Its pivot relation is **Medium**. Since **Track** occurs only once in $path(\Psi_8)$, the affected pivot tuples for the deletion phase are:

$$A^{-}[\Psi_8](u) = \{p^* \in Medium(\sigma_1) \mid \exists d \in D, \exists (s, p, o) \in \Psi_8[r \leftarrow p^*, f \leftarrow d](\sigma_1)\}$$

Because both deleted tuples satisfy $mid = m1$, we obtain:

$$A^{-}[\Psi_8](u) = \{m1\}$$

However, the RDF contribution of $m1$ under Ψ_8 is unchanged after the update, since $m1$ remains connected to the same track identifiers $t1$ and $t2$. Hence:

$$\Delta^{-}[\Psi_8](u) = \emptyset$$

Final deletion set. Collecting the contributions:

$$\Delta^{-}(u) = \Delta^{-}[\Psi_5](u) \cup \Delta^{-}[\Psi_{12}](u) \cup \Delta^{-}[\Psi_7](u) \cup \Delta^{-}[\Psi_8](u)$$

that is,

$$\begin{aligned} \Delta^{-}(u) = \{ & (mbz : t1, rdf : type, mo : Track, \Psi_5), \\ & (mbz : t2, rdf : type, mo : Track, \Psi_5), \\ & (mbz : t1, dc : title, "This Girl", \Psi_{12}), \\ & (mbz : t2, dc : title, "Don't You Know", \Psi_{12}), \\ & (mbz : ga3, foaf:made, mbz : t1, \Psi_7)\} \end{aligned}$$

Phase 2: Computing $\Delta^+(u)$. By Section 6.3,

$$\Delta^+(u) = \bigcup_{\Psi \in \text{Relev}(\text{Track})} \Delta^+[\Psi](u), \quad \Delta^+[\Psi](u) = \Delta_{\text{pivot}}^+[\Psi](u) \cup \Delta_{\text{rel}}^+[\Psi](u).$$

Pivot-relevant rules. For Ψ_5 and Ψ_{12} , the inserted tuples in I are pivot tuples. Thus:

$$\Delta^+[\Psi_5](u) = \{(mbz : t1, rdf : type, mo : Track, \Psi_5), \\ (mbz : t2, rdf : type, mo : Track, \Psi_5)\}$$

and

$$\Delta^+[\Psi_{12}](u) = \{(mbz : t1, dc : title, "This Girl", \Psi_{12}), \\ (mbz : t2, dc : title, "Don't You Know", \Psi_{12})\}$$

Relation-relevant rule Ψ_7 . For the insertion phase, the affected pivot tuples are:

$$A^+[\Psi_7](u) = \{p^* \in \text{Artist}(\sigma_1) \mid \exists i \in I, \exists (s, p, o) \in \Psi_7[r \leftarrow p^*, r_3 \leftarrow i](\sigma_1)\}.$$

Evaluating Ψ_7 with the inserted tuples yields:

$$t'_1 \text{ with } cid = c1 \Rightarrow \text{artists } a1, a2, \quad t'_2 \text{ with } cid = c1 \Rightarrow \text{artists } a1, a2.$$

Hence,

$$A^+[\Psi_7](u) = \{a1, a2\}.$$

For each $p^* \in A^+[\Psi_7](u)$, let

$$S1[\Psi_7](p^*) = \{(s, p, o, g) \mid (s, p, o, g) \in W_0, s = \text{URI}(p^*), g = \text{URI}(\Psi_7)\}$$

and

$$S2[\Psi_7](p^*) = \{(s, p, o, g) \mid (s, p, o) \in \Psi_7[r \leftarrow p^*](\sigma_1), g = \text{URI}(\Psi_7)\}.$$

Then:

$$\Delta_{\text{rel}}^+[\Psi_7](u) = \bigcup_{p^* \in A^+[\Psi_7](u)} (S2[\Psi_7](p^*) \setminus S1[\Psi_7](p^*)).$$

Therefore,

$$\Delta^+[\Psi_7](u) = \{(mbz : ga1, foaf:made, mbz : t1, \Psi_7), \\ (mbz : ga2, foaf:made, mbz : t2, \Psi_7)\}$$

Relation-relevant rule Ψ_8 . For the insertion phase, the affected pivot tuples are:

$$A^+[\Psi_8](u) = \{p^* \in \text{Medium}(\sigma_1) \mid \exists i \in I, \exists (s, p, o) \in \Psi_8[r \leftarrow p^*, f \leftarrow i](\sigma_1)\}$$

Since both inserted tuples still satisfy $mid = m1$, we obtain:

$$A^+[\Psi_8](u) = \{m1\}$$

Again, the RDF contribution of $m1$ under Ψ_8 does not change, and hence:

$$\Delta^+[\Psi_8](u) = \emptyset$$

Final insertion set. Collecting the contributions:

$$\Delta^+(u) = \Delta^+[\Psi_5](u) \cup \Delta^+[\Psi_{12}](u) \cup \Delta^+[\Psi_7](u) \cup \Delta^+[\Psi_8](u)$$

that is,

$$\begin{aligned} \Delta^+(u) = \{ & (mbz : t1, rdf : type, mo : Track, \Psi_5), \\ & (mbz : t2, rdf : type, mo : Track, \Psi_5), \\ & (mbz : t1, dc : title, "This Girl", \Psi_{12}), \\ & (mbz : t2, dc : title, "Don't You Know", \Psi_{12}), \\ & (mbz : ga1, foaf : made, mbz : t1, \Psi_7), \\ & (mbz : ga2, foaf : made, mbz : t2, \Psi_7) \} \end{aligned}$$

Discussion. The computation makes explicit that the affected sets differ between the deletion and insertion phases for Ψ_7 :

$$\begin{aligned} A^-[\Psi_7](u) &= \{a1, a2, a3\} \\ A^+[\Psi_7](u) &= \{a1, a2\} \end{aligned}$$

whereas for Ψ_8 both phases affect the same pivot tuple,

$$A^-[\Psi_8](u) = A^+[\Psi_8](u) = \{m1\}$$

but no net change is produced. Thus, the only effective semantic change in the example is the replacement of the obsolete **foaf:made** assertion for $a3$ by the new assertions induced by the reassignment of **cid**.

5.6 LLM-Compilability as a Consequence of Correctness and Structure

Beyond semantic correctness, the proposed framework exhibits structural properties that make it inherently amenable to LLM-based synthesis.

The correctness result (Theorem 1) establishes that the incremental changeset fully captures the effect of an update on the RDB2RDF view. This provides a

sound and complete specification of the update semantics. However, correctness alone does not guarantee that such a specification can be effectively translated into executable procedures.

A key observation of this work is that LLM-compilability arises from the combination of correctness with specific structural properties of the formalization.

First, the incremental formulation is **compositional**: both $\Delta^{-inc}(u)$ and $\Delta^{+inc}(u)$ are defined as unions of pivot-based and path-based contributions. This enables the synthesis process to decompose the problem into smaller, independently computable units.

Second, the framework exhibits **locality**: the effect of an update is determined solely by the modified relation R and its participation in transformation rules. This eliminates the need for global reasoning over the entire database state.

Third, and most critically, the framework ensures **deterministic derivations**. By Lemma 1, each RDF quad is associated with a unique pivot tuple and transformation rule. This property removes ambiguity in the mapping between input tuples and output quads, which is essential for reliable code generation.

These properties collectively enable a systematic translation from formal definitions to executable artifacts. In particular, large language models can (i) derive abstract algorithms that operationalize the definitions, and (ii) generate concrete trigger implementations that compute the incremental changeset.

This leads to a broader methodological insight: formal data management frameworks can be designed not only for semantic correctness, but also for LLM-compilability. In this perspective, the formal specification serves as a compilation contract, ensuring that any generated implementation preserves the intended semantics.

To the best of our knowledge, this work is among the first to explicitly connect formal correctness, structural properties of specifications, and LLM-based compilation in the context of data management.

A central design goal of the proposed formulation is to enable *LLM-compilability*, i.e., the ability to systematically translate formal definitions into correct and minimal executable implementations using large language models.

The refined formulation of the incremental deletion component achieves this by decomposing the computation into *independent contributions per pivot tuple*. More precisely, instead of requiring a global evaluation of a transformation rule over the entire database state, the computation is structured around the set of relevant pivot tuples:

$$A = \{p^* \in R^*(\sigma_1) \mid \exists d \in D \text{ such that } p^* \in P[\Psi](d; \sigma_1)\}.$$

For each $p^* \in A$, the incremental deletion contribution is computed locally as $\Delta^-[\Psi](p^*, \sigma_1)$, and the final result is obtained by aggregation.

This decomposition has important implications for LLM-based synthesis: Each contribution $\Delta^-[\Psi](p^*, \sigma_1)$ can be computed independently, given a binding for p^* and the deleted tuples $d \in D$. This avoids the need for global reasoning over all tuples in the pivot relation and aligns with the localized inference patterns that LLMs handle more reliably.

6 Operationalization of the Formal Model

This section is structured in three parts. Section 6.1 delves into the motivation for creating an operationalization of the model. Section 6.2 introduces an asynchronous architecture that re-engineers Algorithm 1 to separate access to the post-update database state σ_1 from access to the pre-update view state W_0 , since the database and the view are typically in distinct environments. Section 6.3 then describes the LLM-based prototype translation workflow that produces the required artifacts to maintain the view incrementally.

6.1 Motivation

A natural question arising from the previous sections is whether the formal model can be implemented in practice. To answer this question, this section introduces an LLM-based prototype translation workflow that starts with R2RML mappings that define an entity-preserving RDB2RDF view and produces the required AFTER triggers and related artifacts to maintain the view incrementally.

It should be stressed that the goal is not to introduce a production-ready compiler or a new LLM code-generation method. Instead, an LLM is used as an experimental instrument to investigate whether the formal model is sufficiently precise and constructive to guide the generation of executable incremental-maintenance logic. The LLM is not used to establish the correctness of the proposed framework, which follows from the formal results.

The prototype serves two complementary objectives.

The first objective is to demonstrate the practical realizability of the proposed compilation theory. Given the formal definitions introduced in this paper and an Entity-Preserving R2RML mapping, the compiler automatically generates executable PostgreSQL triggers that implement the incremental maintenance algorithm. Successful compilation and execution provide experimental evidence that the proposed formal framework is sufficiently precise, complete, and operational to be translated into executable database code.

The second objective is to investigate a broader research question: *Can a sufficiently precise formal database theory be directly operationalized by a Large Language Model?* Rather than asking an LLM to generate isolated pieces of code, the proposed compiler requires the model to interpret formal definitions, reason about semantic constraints, identify pivot relations, derive URI constructors, construct intermediate semantic representations, and consistently apply the compilation rules defined by the framework. In this sense, the prototype evaluates the feasibility of using LLMs not merely as code generators, but as semantic compilation engines capable of instantiating complex formal theories into executable systems.

Accordingly, the prototype constitutes both an implementation of the proposed framework and an experimental validation of its compilability. If the generated triggers execute correctly and produce exactly the RDF changes predicted by the formal semantics, then the compilation model can be considered operationally

realizable. At the same time, these results demonstrate that current LLMs can compile a complex formal database theory into correct executable code.

6.2 An Asynchronous Architecture

Algorithm 1 defines how to compute the changesets $\Delta^-(u)$ and $\Delta^+(u)$, but it must be re-engineered to separate accessing the post-update database state σ_1 from accessing the pre-update view state W_0 , since the database and the view are typically in distinct environments. This section then explores an architecture, shown in Figure 4, that addresses this problem.

The first step is to rewrite the definition of $\Delta^-(u)$ as follows. First define:

$$\begin{aligned}
 S_1[\Psi](u) &= \bigcup_{p^* \in A^-[\Psi](u)} S_1[\Psi](p^*) \\
 &= \bigcup_{p^* \in A^-[\Psi](u)} \{(s, p, o, g) \mid (s, p, o, g) \in W_0, s = URI(p^*), g = URI(\Psi)\} \\
 S_2[\Psi](u) &= \bigcup_{p^* \in A^-[\Psi](u)} S_2[\Psi](p^*) \\
 &= \bigcup_{p^* \in A^-[\Psi](u)} \{(s, p, o, g) \mid (s, p, o) \in \Psi[r^* \leftarrow p^*](\sigma_1), g = URI(\Psi)\}
 \end{aligned}$$

The following proposition shows that $\Delta_{rel}^-[\Psi](u)$ can be redefined using $S_1[\Psi](u)$, computed on W_0 and $A^-[\Psi](u)$, and $S_2[\Psi](u)$, computed on σ_1 and $A^-[\Psi](u)$.

Proposition 1. *Let $\mathcal{W} = (V, \mathbf{S}, M)$ be an RDB2RDF view. Assume that $\mathcal{W}$ is entity-preserving. Then,*

$$\Delta_{rel}^-[\Psi](u) = S_1[\Psi](u) \setminus S_2[\Psi](u)$$

(The proof can be found in Appendix A).

Recall that:

$$\begin{aligned}
\Delta_{pivot}^-[\Psi](u) &= \bigcup_{d \in D} \Delta_{pivot}^-[\Psi](d) \\
&= \bigcup_{d \in D} \{(s, p, o, g) \mid (s, p, o, g) \in W_0, s = URI(d), g = URI(\Psi)\} \\
A^-[\Psi](u) &= \{p^* \in R^*(\sigma_1) \mid \exists d \in D, \exists r_k \in VarR(\Psi), \\
&\quad \exists (s, p, o) \in \Psi[r^* \leftarrow p^*, r_k \leftarrow d](\sigma_1[R \leftarrow ((R \cup D) \setminus I]))\} \\
S_1[\Psi](u) &= \bigcup_{p^* \in A^-[\Psi](u)} \{(s, p, o, g) \mid (s, p, o, g) \in W_0, s = URI(p^*), g = URI(\Psi)\} \\
S_2[\Psi](u) &= \bigcup_{p^* \in A^-[\Psi](u)} \{(s, p, o, g) \mid (s, p, o) \in \Psi[r^* \leftarrow p^*](\sigma_1), g = URI(\Psi)\} \\
\Delta_{rel}^-[\Psi](u) &= S_1[\Psi](u) \setminus S_2[\Psi](u) \\
\Delta^-[\Psi](u) &= \Delta_{pivot}^-[\Psi](u) \cup \Delta_{rel}^-[\Psi](u)
\end{aligned}$$

and

$$\begin{aligned}
\Delta_{pivot}^+[\Psi](u) &= \bigcup_{i \in I} \Delta_{pivot}^+[\Psi](i) \\
&= \bigcup_{i \in I} \{(s, p, o, g) \mid (s, p, o) \in \Psi[r \leftarrow i](\sigma_1), g = URI(\Psi)\} \\
A^+[\Psi](u) &= \{p^* \in R^*(\sigma_1) \mid \exists i \in I, \exists r_k \in VarR(\Psi), \\
&\quad \exists (s, p, o) \in \Psi[r^* \leftarrow p^*, r_k \leftarrow i](\sigma_1)\} \\
\Delta_{rel}^+[\Psi](u) &= \bigcup_{p^* \in A^+[\Psi](u)} \Delta_{rel}^+[\Psi](p^*) \\
&= \bigcup_{p^* \in A^+[\Psi](u)} \{(s, p, o, g) \mid (s, p, o) \in \Psi[r^* \leftarrow p^*](\sigma_1), g = URI(\Psi)\} \\
\Delta^+[\Psi](u) &= \Delta_{pivot}^+[\Psi](u) \cup \Delta_{rel}^+[\Psi](u)
\end{aligned}$$

Let $u = (D, I)$ be an update, W_0 be the pre-update view state, and σ_1 be the post-update database state. Then, note that:

- $\Delta_{pivot}^-[\Psi](u)$: defined using W_0 and u .
- $A^-[\Psi](u)$: defined using σ_1 and u .
- $S_1[\Psi](u)$: defined using W_0 and $A^-[\Psi](u)$.
- $S_2[\Psi](u)$: defined using σ_1 and $A^-[\Psi](u)$.
- $\Delta^-[\Psi](u)$: defined using W_0 , σ_1 , and u .
- $\Delta_{pivot}^+[\Psi](u)$: defined using σ_1 and u .
- $A^+[\Psi](u)$: defined using σ_1 and u .
- $\Delta_{rel}^+[\Psi](u)$: defined using σ_1 and $A^+[\Psi](u)$.
- $\Delta^+[\Psi](u)$: defined using σ_1 and u .

The architecture combines AFTER triggers with an asynchronous maintenance worker. Instead of directly computing and applying the changeset inside a relational transaction, a trigger captures a snapshot of the update event on a relation R (the transition tables D and I), and computes the affected tuples for deletion $A^-[\Psi](u)$, the post-update contributions $S_2[\Psi](u)$ to $\Delta^-(u)$, and the insertion changeset $\Delta^+(u)$ (which requires computing the affected tuples for insertion $A^+[\Psi](u)$). This information is then stored in a maintenance queue. The AFTER trigger for a relation R calls the function `COMPUTE_CHANGESET_R`, shown in Algorithm 2, to compute $A^-[\Psi](u)$, $S_2[\Psi](u)$, and $\Delta^+[\Psi](u)$ and update the maintenance queue.

An external worker processes queued events sequentially, preserving the update commit order. For each event, the worker first queries the triplestore to retrieve from the RDB2RDF view state W_0 the corresponding deletion contribution of deleted tuples, $\Delta_{pivot}^-[\Psi](u)$, and the pre-update RDF contributions, $S_1[\Psi](u)$. Then, it computes the deletion contribution using the set difference $(S_1[\Psi](u) \setminus S_2[\Psi](u))$, applies $\Delta^-(u)$ to the view, and then applies $\Delta^+(u)$.

This architecture offers several advantages. First, it avoids executing SPARQL queries inside relational transactions, preventing triplestore latency or temporary unavailability from blocking database updates. Second, it preserves correctness by storing the relational context of the update at commit time, ensuring that subsequent database updates do not affect the computation of the changeset. Third, the queue-based design improves scalability, fault tolerance, and recoverability because maintenance events can be retried or processed asynchronously without affecting transactional workloads.

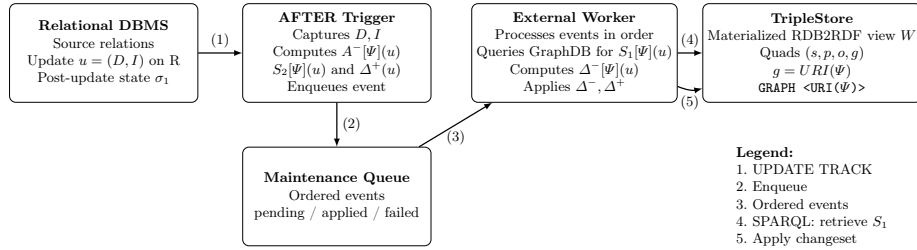


Fig. 4. Asynchronous architecture for incremental maintenance of a materialized RDB2RDF view. AFTER triggers capture the transition tables D and I , and compute $A^-[\Psi](u)$, $S_2[\Psi](u)$, and $\Delta^+[\Psi](u)$ (which requires computing $A^+[\Psi](u)$). An external worker processes queued events in commit order, retrieves $\Delta_{pivot}^-[\Psi](u)$ and $S_1[\Psi](u)$ from the triplestore, computes $\Delta^-[\Psi](u)$, and updates the RDF materialization.

Algorithm 2: Function COMPUTE_CHANGESET_R for the Asynchronous Architecture

Input: RDB2RDF view $W = (V, S, M)$; update $u = (D, I)$ over relation R .
Output: Insertions on tables *rdm_maintenance_queue* and *rdm_rule_contribution*.

// Event-level queue initialization:

1. Insert one row into *rdm_maintenance_queue* for the statement-level update $u = (D, I)$;
- // Rule-level computation and publication:
2. **foreach** $\Psi \in \text{Relev}(R)$ **do**
 - // Execute the following two steps sequentially:
 - // STEP 1 - Incremental Maintenance Computation for Ψ
 3. $\Delta_{pivot}^+[\Psi](u) \leftarrow \emptyset$;
 4. $\Delta_{rel}^+[\Psi](u) \leftarrow \emptyset$;
 5. **if** Ψ is pivot-relevant to updates on R **then**
 6. | Compute $\Delta_{pivot}^+[\Psi](u)$;
 7. **end**
 8. **if** Ψ is relation-relevant to updates on R **then**
 9. | Compute $A^-[\Psi](u)$, $A^+[\Psi](u)$, $S_2[\Psi](u)$, and $\Delta_{rel}^+[\Psi](u)$;
 10. **end**
 11. $\Delta^+[\Psi](u) \leftarrow \Delta_{pivot}^+[\Psi](u) \cup \Delta_{rel}^+[\Psi](u)$;
 - // STEP 2 - Rule Contribution Publication
 12. Insert one row into *rdm_rule_contribution* for Ψ with $A^-[\Psi](u)$, $S_2[\Psi](u)$, and $\Delta^+[\Psi](u)$
 13. | linked to the *event_id* created in *rdm_maintenance_queue*;
 14. **end**

6.3 An LLM-based Prototype Translation Workflow

The compiler follows a two-phase compilation process. Each compilation phase is implemented by a specialized LLM-based compiler module that generates a well-defined intermediate artifact. Since LLMs may occasionally produce incomplete or inconsistent outputs, each generated artifact is independently verified before being accepted as the input of the subsequent compilation phase. This verification process ensures that every compilation step conforms to the formal rules defined by the proposed framework.

Phase 1 – Translating R2RML Statements into Transformation Rules

The first phase translates an Entity-Preserving R2RML mapping into a validated set of transformation rules. An LLM-assisted transformation rules translation module implements this phase, guided by a prompt (available in the supplemental material), whose inputs consist of the relational database schema and the R2RML mapping.

This module performs the following tasks:

1. Identifies pivot relations and verifies entity preservation.
2. Derives URI constructors and the corresponding hasURI predicates.
3. Validates the pivot relations and URI constructors.
4. Generates and validates the transformation rules.

The output is a structured JSON specification representing a certified intermediate artifact. For each R2RML Triples Map, the specification records:

- the entity-preservation analysis;
- the identified pivot relation;
- the derived URI constructor;
- the corresponding hasURI predicate;
- the validation results; and
- the generated Transformation Rules.

Mappings that fail validation are reported together with the detected inconsistencies and marked as “Manual Analysis Required”. No Transformation Rules are generated for invalid mappings.

This intermediate representation completely separates semantic reasoning from code generation. Consequently, the second compilation phase no longer needs to interpret the R2RML mapping and instead relies exclusively on the certified semantic specification.

Phase 2 – Translating Transformation Rules into AFTER Triggers

The second phase translates the transformation rules generated during Phase 1 into AFTER triggers, as discussed in Section 6.2. An LLM-assisted trigger translation module implements this phase, guided by a prompt (available at the supplemental material), whose inputs consist of the transformation rules

generated during Phase 1, and the incremental maintenance framework of Section 6.2.

Each trigger captures a snapshot of the update event on a relation R (the transition tables D and I), and computes the affected tuples for deletion $A^-[\Psi](u)$, the post-update contributions $S2[\Psi](u)$ to $\Delta^-(u)$, and the insertion changeset $\Delta^+(u)$ (which requires computing the affected tuples for insertion $A^+[\Psi](u)$). The resulting code operates on the relational database and communicates with the asynchronous maintenance worker through the queue, without directly accessing the materialized RDF view.

7 Operational Feasibility Study

The goal of this section is to investigate the operationalization of the formal model proposed in Section 6. As a case study, it adopts the LinkedBrainz RDB2RDF View, defined on top of the MusicBrainz relational database using a set of R2RML statements, and validates the complete cycle from R2RML statements to changeset computation.

The goal is specialized into the following questions:

- RQ1 — Correctness of the Transformation Rules Translation:** Can R2RML statements be correctly translated into transformation rules?
- RQ2 — Correctness of the AFTER Triggers Translation:** Can transformation rules be correctly translated into executable AFTER triggers and the maintenance worker?
- RQ3 — End-to-End Correctness:** Do the generated changesets specified by the transformation rules produce the same RDF state as a complete recomputation?
- RQ4 — Human Intervention:** Which parts of the operationalization require human decisions, and why?

The code, database, and RDB2RDF view adopted in the case study are available at <https://github.com/semantic-ekgraphs/IVM>.

7.1 Setup

Data and LLMs. The experiments were based on:

- *Relational Database:* a local copy of the *MusicBrainz* database, stored in PostgreSQL.
- *LinkedBrainz RDB2RDF View:* an RDB2RDF view of the MusicBrainz database, materialized in GraphDB.
- *Selected relations:* the relations *ARTIST*, *MEDIUM*, and *TRACK*, chosen because they occur in complex transformation rules.
- *Selected updates:* one insertion, one deletion, and one update per selected relation, with the expected changesets.

- *Selected LLMs*: GPT-5.6 using the CODEX desktop, and Qwen3.8-Max-Preview in “auto” mode using the free-tier chat interface, referred to simply as GPT-5.6 and Qwen3.8 in what follows.

MusicBrainz³ is a large and actively maintained open music knowledge base operated by the MetaBrainz Foundation. The relational database schema comprises 106 relations, including data on artists, release groups, and related entities. The database is implemented in PostgreSQL and continuously publishes incremental database updates through its replication mechanism, providing a realistic scenario of high-frequency relational changes. This makes the case study particularly suitable for evaluating incremental maintenance techniques for materialized RDB2RDF views.

The LinkedBrainz RDB2RDF view⁴ uses the Music Ontology vocabulary and is specified in R2RML through 57 TriplesMaps⁵. Although MusicBrainz already distributes incremental relational updates through replication packets, maintaining a materialized view over MusicBrainz still requires computing changesets that reflect the impact of relational updates on the RDF materialized view.

7.2 Correctness Validation Procedure

Transformation Rules Translation Correctness

This step addresses RQ1 and, in part, RQ4. GPT-5.6 was prompted to translate the R2RML statements of the case study into transformation rules. Then, the results were verified, separating URI constructor from transformation rules translation correctness validation. GPT-5.6 also helped in iteratively refining the translation prompt until correct, traceable transformation rules were produced.

URI constructor correctness validation checks the preservation of the complete subject template, placeholder expressions, and ordering, as well as encoding semantics and consistency across mappings representing the same semantic entity. Transformation rules translation correctness validation checks the RDF class or property, RDF term type, selection conditions, value transformations, null handling, and relational paths.

When the R2RML input contains ambiguities, conflicting URI templates, malformed SQL expressions, unresolved relational paths, or potentially invalid IRI values, the translation phase still generates a candidate rule and marks it for human validation. The output includes the supporting evidence and the issue requiring review. A candidate is validated only after passing all automatic checks or after the corresponding human decision is explicitly recorded.

³ Available at <https://musicbrainz.org>

⁴ Available at https://musicbrainz.org/doc/LinkedBrainz/Software_Approach

⁵ Available at <https://docs.openlinksw.com/virtuoso/rdfviewsbusintmbr/>

AFTER Trigger Translation Correctness

This step addresses RQ2 and, in part, RQ4. GPT-5.6 and Qwen3.8 were prompted to translate the transformation rules for the selected relations – **ARTIST**, **MEDIUM**, and **TRACK** – into AFTER Triggers.

For each combination of LLM and selected relation, the correctness of the generated trigger code was manually validated according to the criteria presented in Table 4.

Table 4. Trigger Translation and End-to-End Correctness.

Criteria	Description
<i>AFTER Trigger Translation Correctness</i>	
Transformation Rule Identification	Does the LLM-assisted trigger translation module correctly identify the relevant transformation rules?
Rule Contribution Computation	Does the LLM-assisted trigger translation module correctly generate the code for the rule contributions?
Code Minimality	Does the LLM-assisted trigger translation module generate code which is concise and free of unnecessary operations?
Hallucination	Does the LLM-assisted trigger translation module introduce unsupported assumptions or irrelevant joins?
<i>End-to-End Correctness</i>	
Rule Contribution Correctness	Do the generated triggers compute the expected rule contributions for each relevant transformation rule and update scenario?
Changeset Correctness	Do the generated triggers compute the expected changesets for each update scenario?

End-to-End Correctness

This step addresses RQ3. The AFTER triggers for the selected relations were executed using different update scenarios. To be more informative, correctness validation was subdivided into two tasks.

Rule Contribution Correctness. Rule contribution correctness validation verifies whether the AFTER triggers compute the expected maintenance data for each relevant transformation rule and update scenario.

In more detail, for each update u and each relevant transformation rule $\Psi \in \text{Relev}(R)$, the generated rule contributions

$$A^-[\Psi], A^+[\Psi], S2[\Psi], \Delta_{pivot}^+[\Psi], \Delta_{rel}^+[\Psi], \Delta^+[\Psi]$$

were manually compared against the expected results derived from the formal incremental maintenance semantics.

Changeset Correctness. This step fully addresses RQ3 by providing an end-to-end correctness validation of the generated maintenance code. It verifies whether the generated changesets produce the same RDF view as a full recomputation, which is independently obtained.

Let W_0 denote the RDF view before the update and $W_1 = M(\sigma_1)$ the RDF view obtained by applying the complete RDB2RDF mapping to the updated

database state σ_1 . The generated triggers are considered correct iff the RDF view reconstructed from their computed changesets satisfies $W_1 = (W_0 - \Delta^-(u)) \cup \Delta^+(u)$.

7.3 Results

Transformation Rules Translation Correctness

All R2RML statements of the case study were classified as entity-preserving and translated into 89 transformation rules (TRs). The correctness of 78 TRs was automatically validated, but 11 (12.4%) required human validation, primarily due to malformed SQL joins or mapping inconsistencies.

AFTER Trigger Translation and End-to-End Correctness

Table 5 summarizes the AFTER trigger translation and end-to-end correctness validation for the **TRACK** relation, with the triggers obtained with GPT-5.6 and Qwen3.8. Tables 6 and 7 summarize the validation for **ARTIST** and **MEDIUM**, with the triggers obtained with GPT-5.6. The supplemental material contains the complete validation, organized as in the `readme.md` file.

Summary

To summarize, the results suggest that:

- GPT-5.6 correctly translated all R2RML statements into transformation rules, but some rules required human validation.
- GPT-5.6 generated correct and optimized AFTER trigger code in most cases;
- Qwen 3.8-Max-Preview generated correct and concise AFTER trigger code in some cases.

8 Conclusions

The first contribution of this paper was a formal model of the deletion and insertion components of RDB2RDF changesets. Existing formulations of incremental RDB2RDF maintenance depend on the pre-update relational state when computing deletion changesets. This creates a mismatch with statement-level AFTER triggers, which execute when the database is already in its post-update state and have direct access only to the transition tables and the current database state. This paper introduces a formal model that removes this mismatch through a post-update formulation and proves the correctness of the resulting changesets.

The second contribution was to describe an LLM-based prototype translation workflow that starts with R2RML statements that define an entity-preserving RDB2RDF view and produces the required AFTER triggers and related artifacts to maintain the view incrementally. The prototype provides evidence of the operational adequacy of the formal model.

Table 5. Trigger Translation and End-to-End Correctness Validation – Results for Relation **Track** – GPT-5.6 and Qwen3.8.

Criteria	Model	Result
<i>AFTER Trigger Translation Correctness</i>		
Transformation rule Identification	GPT	Identified all relevant rules.
	Qwen	Identified all relevant rules.
Rule Contribution Computation	GPT	Correctly computed all rule contributions.
	Qwen	Correctly computed all rule contributions.
Code Minimality	GPT	Produced a redundant intermediate code.
	Qwen	Produced a redundant intermediate code.
Hallucination	GPT	Did not hallucinate.
	Qwen	Did not hallucinate.
<i>End-to-End Correctness</i>		
Rule Contribution Correctness	GPT	The triggers generated contributions for all 7 rules evaluated. The generated quadruples are correct for 5 of the rules, while the remaining 2 rules show discrepancies regarding literal datatypes.
	Qwen	Of the contributions generated by the 7 rules, 4 produced correct quadruples. A fifth rule was considered correct with the inclusion of <i>xsd:string</i> . The other 2 differ only in the data types of the literals
Changeset Correctness	GPT	The computed changesets correspond exactly to the expected changesets.
	Qwen	(Failed to execute)

Table 6. Trigger Translation and End-to-End Correctness Validation – Results for Relation **Medium** – GPT-5.6 only.

Criteria	Result
<i>AFTER Trigger Translation Correctness</i>	
Transformation Rule Identification	Identified all relevant rules.
Rule Contribution Computation	Correctly produced all relational joins.
Code Minimality	Produced minimal code.
Hallucination	Did not hallucinate.
<i>End-to-End Correctness</i>	
Rule Contribution Correctness	The trigger on medium relation generated contributions for all 6 rules evaluated. The produced quadruples are correct for all of the rules.
Changeset Correctness	The computed changesets correspond exactly to the expected changesets.

Table 7. Trigger Translation and End-to-End Correctness Validation – Results for Relation **Artist** – GPT-5.6 only.

Criteria	Result
<i>AFTER Trigger Translation Correctness</i>	
Transformation Rule Identification	Identified all relevant rules.
Rule Contribution Computation	Correctly produced all relational joins.
Code Minimality	Relatively clean and concise.
Hallucination	Did not hallucinate.
<i>End-to-End Correctness</i>	
Rule Contribution Correctness	Out of the 14 rules, the quadruples for 13 of them were generated correctly. The 14th rule generated quadruples with a URI encoding error due to a whitespace character.
Changeset Correctness	The computed changesets correspond exactly to the expected changesets.

The third contribution was to conduct an experiment using a real-world database and an RDB2RDF view specified with R2RML statements, illustrating that an LLM, guided by appropriate prompts and by accessing the formal model, can generate code to compute and publish the changesets.

As future work, the LLM-based prototype translation workflow described in Section 6.3 will be re-engineered as a multi-agent system with human-in-the-loop to facilitate compiling R2RML statements into AFTER triggers and installing them in the source relational database.

Acknowledgements. Anonymized.

Supplemental Material Statement. The extended paper, with the formal definitions, results, and proofs, and the code, prompts, database, and RDB2RDF view adopted in the experiments are available at <https://github.com/semantic-ekgraphs/IVM.git>

Declaration of use of Generative AI. In Section 5, ChatGPT was used to help check the definitions. In Section 6, ChatGPT was used to help define the architecture. In Section 7, GPT-5.6 and Qwen3.8 were utilized to generate code.

Appendix A. Proofs

In what follows, let $\mathcal{W} = (V, \mathbf{S}, M)$ be an RDB2RDF view, $u = (D, I)$ be an update on a relation R , σ_0 and σ_1 be the database states before and after applying u , and $W_1 = \text{RDF_State}(\mathcal{W}, \sigma_1)$ and $W_0 = \text{RDF_State}(\mathcal{W}, \sigma_0)$ be the view states before and after applying u . Let $\Delta^-(u)$ and $\Delta^+(u)$ be the deletion and insertion components defined as in Section 5.

Let Ψ be a rule in M . Recall from Section 5.2 that

$$\Delta_{rel}^-[\Psi](p^*) = (S_1[\Psi](p^*) \setminus S_2[\Psi](p^*))$$

Also recall that the overall incremental deletion contribution of a rule $\Psi \in \mathcal{M}$ for an update $u = (D, I)$ on a relation R , when Ψ is relation-relevant to R , is defined by aggregating the contributions of all affected pivot tuples:

$$\Delta_{rel}^-[\Psi](u) = \bigcup_{p^* \in A^-[\Psi](u)} \Delta_{rel}^-[\Psi](p^*)$$

Also recall that

$$S_1[\Psi](u) = \bigcup_{p^* \in A^-[\Psi](u)} S_1[\Psi](p^*)$$

$$S_2[\Psi](u) = \bigcup_{p^* \in A^-[\Psi](u)} S_2[\Psi](p^*)$$

Proposition 1. *Let $\mathcal{W} = (V, \mathbf{S}, M)$ be an RDB2RDF view. Assume that $\mathcal{W}$ is entity-preserving. Then,*

$$\Delta_{rel}^-[\Psi](u) = S_1[\Psi](u) \setminus S_2[\Psi](u)$$

Proof. Let p and q be two distinct pivot tuples of Ψ . Since, by assumption, $\mathcal{W}$ is entity-preserving, it follows that $URI(p) \neq URI(q)$. Hence, from the definition of $S_1[\Psi](p)$ and $S_2[\Psi](q)$, it follows that, for $m, n \in \{1, 2\}$, with $m \neq n$:

$$S_m[\Psi](p) \cap S_n[\Psi](q) = \emptyset$$

which implies that

$$\begin{aligned} \Delta_{rel}^-[\Psi](u) &= \bigcup_{p^* \in A^-[\Psi](u)} \Delta_{rel}^-[\Psi](p^*) \\ &= \bigcup_{p^* \in A^-[\Psi](u)} (S_1[\Psi](p^*) \setminus S_2[\Psi](p^*)) \\ &= \left(\bigcup_{p^* \in A^-[\Psi](u)} S_1[\Psi](p^*) \right) \setminus \left(\bigcup_{p^* \in A^-[\Psi](u)} S_2[\Psi](p^*) \right) \\ &= S_1[\Psi](u) \setminus S_2[\Psi](u) \end{aligned}$$

□

Let σ be a relational database state. Recall that $\mathcal{P}(\mathcal{W})$ denotes the set of pivot relations of $\mathcal{W}$. By Definition 3, the RDF state of $\mathcal{W}$ under σ is given by:

$$RDF_State(\mathcal{W}, \sigma) = \bigcup_{R^* \in \mathcal{P}(\mathcal{W})} \bigcup_{p^* \in R^*(\sigma)} RDF_State[R^*](p^*, \sigma)$$

The first lemma shows that each quad has a unique derivation associated with a single pivot tuple and transformation rule.

Lemma 1 (Uniqueness of Quad Generation).

For every quad $(s, p, o, g) \in RDF_State(\mathcal{W}, \sigma)$, there exists a unique transformation rule $\Psi \in M$ and a unique pivot tuple $p^ \in pivot(\Psi)(\sigma)$ such that*

$$(s, p, o, g) \in RDF_State[\Psi](p^*, \sigma)$$

Proof. By definition of $RDF_State(\mathcal{W}, \sigma)$, every quad (s, p, o, g) is generated by applying some transformation rule $\Psi \in M$ to some pivot tuple $p^* \in pivot(\Psi)(\sigma)$. Thus, existence is immediate.

We now prove uniqueness.

Suppose that a quad (s, p, o, g) is generated by two pairs (Ψ_1, p_1) and (Ψ_2, p_2) , i.e.,

$$(s, p, o, g) \in RDF_State[\Psi_1](p_1, \sigma) \quad \text{and} \quad (s, p, o, g) \in RDF_State[\Psi_2](p_2, \sigma)$$

We show that $\Psi_1 = \Psi_2$ and $p_1 = p_2$.

Assume $\Psi_1 \neq \Psi_2$.

By construction of the RDF state, the provenance component g is defined as $g = URI(\Psi)$. Therefore, quads generated by different transformation rules have different provenance identifiers. Hence, (s, p, o, g) cannot be generated by both Ψ_1 and Ψ_2 , which contradicts the assumption. Therefore, $\Psi_1 = \Psi_2$, which contradicts the assumption of the case.

Assume $\Psi_1 = \Psi_2 = \Psi$, but $p_1 \neq p_2$.

By the entity-preserving assumption, distinct pivot tuples are mapped to distinct RDF subjects. Therefore, all triples generated from p_1 have $URI(p_1)$ as subject, and all triples generated from p_2 have $URI(p_2)$ as subject, with

$$URI(p_1) \neq URI(p_2)$$

Thus, no triple (and hence no quad) can be generated from both p_1 and p_2 . This contradicts the assumption that (s, p, o, g) belongs to both $RDF_State[\Psi](p_1, \sigma)$ and $RDF_State[\Psi](p_2, \sigma)$. Thus, $p_1 = p_2$, which contradicts the assumption of the case.

Therefore, $\Psi_1 = \Psi_2$ and $p_1 = p_2$, which proves uniqueness.

□

□

The next two lemmas show that the incremental deletion and the incremental insertion components capture the quads whose derivations are invalidated by deletions or created by insertions.

Recall that a quad q has a derivation under state σ iff there is a rule Φ , with pivot relation T^* , and a pivot tuple $t^* \in T^*(\sigma)$ such that $q \in \Phi[r \leftarrow t^*](\sigma)$.

Lemma 2 (Characterization of the Deletion Component).

- (a) (For any quad q) $((q \in W_0 \wedge q \notin \Delta^-(u)) \Rightarrow q \in W_1)$.
- (b) (For any quad q) $(q \in W_1 \Rightarrow (q \notin \Delta^-(u) \vee q \in \Delta^+(u)))$.

Proof

Part (a): We first note that Part (a) is equivalent to

$$(\text{For any quad } q)((q \in W_0 \wedge q \notin W_1) \Rightarrow q \in \Delta^-(u))$$

Assume that $q \in W_0$ and $q \notin W_1$. We prove that $q \in \Delta^-(u)$.

Since $q = (s, p, o, g) \in W_0$, there exist a transformation rule $\Psi \in M$ and a pivot tuple $p^* \in \text{pivot}(\Psi)(\sigma_0)$ such that

$$(s, p, o) \in \Psi[r^* \leftarrow p^*](\sigma_0) \quad \text{and} \quad g = \text{URI}(\Psi)$$

Let $u = (D, I)$ be the update over relation R . There are two cases to consider.

Case 1: $p^* \in D$.

Then, the definition of $\Delta_{\text{pivot}}^-(\Psi)$ in Section 4.2.1 applies to p^* :

$$\Delta_{\text{pivot}}^-(\Psi)(p^*) = \{(s, p, o, g) \mid (s, p, o, g) \in W_0, s = \text{URI}(p^*), g = \text{URI}(\Psi)\}$$

Since $q = (s, p, o, g) \in W_0$, q is derived from p^* using Ψ in σ_0 , and $p^* \in D$, it follows that:

$$q \in \Delta_{\text{pivot}}^-(\Psi)(p^*) \subseteq \Delta^-(u)$$

Case 2: $p^* \notin D$.

Then, since $q \notin W_1$, there must be an occurrence of R in $\text{path}(\Psi)$ such that this occurrence is not the pivot relation of ψ (that is, the occurrence of R is not the first occurrence of a relation scheme in $\text{path}(\Psi)$) and this occurrence is affected by the deletions in $u = (D, I)$. Hence, ψ is relation-relevant to R with respect to $u = (D, I)$ and the definitions in Section 4.2.2 apply to R . Let R^* be the pivot relation of Ψ .

Since $q \notin W_1$, the pivot tuple p^* that generated q in σ_0 using Ψ is affected by the deletion of tuples in D , and therefore

$$p^* \in A^-(\Psi)(u)$$

But, by assumption, $q \in W_0$ and q is derived using the pivot tuple p^* and the rule Ψ in σ_0 . Hence, by definition of $S_1[\Psi](p^*)$, it follows that

$$q \in S_1[\Psi](p^*)$$

Also, by assumption, $q \notin W_1$. Hence, by definition of $S_2[\Psi](p^*)$, it follows that

$$q \notin S_2[\Psi](p^*)$$

Hence,

$$q \in (S_1[\Psi](p^*) \setminus S_2[\Psi](p^*))$$

Therefore, for the pivot tuple p^* , by definition of $\Delta_{rel}^-[\Psi](p^*)$ in Section 4.2.2, it follows that:

$$q \in (S_1[\Psi](p^*) \setminus S_2[\Psi](p^*)) = \Delta_{rel}^-[\Psi](p^*)$$

Hence

$$q \in \Delta_{rel}^-[\Psi](u) \subseteq \Delta^-(u)$$

Thus, in both cases, if $q \in W_0$ and $q \notin W_1$, then $q \in \Delta^-(u)$.

Part (b): We first note that Part (b) is equivalent to

$$(\text{For any quad } q)((q \in \Delta^-(u) \wedge q \notin \Delta^+(u)) \Rightarrow q \notin W_1)$$

Assume that $q \in \Delta^-(u)$ and $q \notin \Delta^+(u)$. We prove that $q \notin W_1$.

By Section 4.1, the deletion component is defined as the union of the deletion contributions of all transformation rules relevant to R :

$$\Delta^-(u) = \bigcup_{\Psi \in \text{Relev}(R)} \Delta^-[\Psi](u)$$

Hence, there exists a transformation rule Ψ relevant to R such that

$$q \in \Delta^-[\Psi](u)$$

There are two cases to consider, according to the decomposition of Section 4.2:

$$\Delta^-[\Psi](u) = \Delta_{pivot}^-[\Psi](u) \cup \Delta_{rel}^-[\Psi](u)$$

Case 1: $q \in \Delta_{pivot}^-[\Psi](u)$.

In this case, Ψ is pivot-relevant to R , that is, $\text{pivot}(\Psi) = R$. Recall that $u = (D, I)$. By Section 4.2.1,

$$\Delta_{pivot}^-[\Psi](u) = \bigcup_{d \in D} \Delta_{pivot}^-[\Psi](d)$$

where each $\Delta_{pivot}^-[\Psi](d)$ is exactly the RDF contribution of a deleted pivot tuple $d \in D$ using Ψ in state σ_0 .

By Lemma 1, the transformation rule Ψ and the pivot tuple $d \in D$ are the unique way of deriving q in σ_0 . In particular, the subject of q is $URI(d)$. Hence, there would be no pivot tuple $t \in (R(\sigma_0) \setminus D)$ such that the subject of q is $URI(t)$.

By Lemma 3, $\Delta^+(u)$ is the set of quads whose derivation becomes valid in σ_1 due to some tuple $i \in I$. But $q \notin \Delta^+(u)$ by assumption. Hence, q has no derivation in σ_1 due to some tuple $i \in I$. In particular, there would be no pivot tuple $t \in I$ such that the subject of q is $URI(t)$.

Now, $R(\sigma_1) = ((R(\sigma_0) \setminus D) \cup I)$. Therefore, there would be no pivot tuple $t \in R(\sigma_1)$ such that the subject of q is $URI(t)$. Hence, q has no derivation in σ_1 , that is, $q \notin W_1$.

Case 2: $q \in \Delta_{rel}^-[\Psi](u)$.

In this case, Ψ is relation-relevant to R , that is, $R \in \text{Relations}(\text{path}(\Psi))$. Let R^* be the pivot relation of Ψ . By Section 4.2.2, there exists an affected pivot tuple $p^* \in R^*(\sigma_0)$ such that

$$p^* \in A^-[\Psi](u)$$

and

$$q \in \Delta_{rel}^-[\Psi](p^*)$$

But, for relation-relevant rules, the deletion contribution of p^* is defined by

$$\Delta_{rel}^-[\Psi](p^*) = (S_1[\Psi](p^*) \setminus S_2[\Psi](p^*))$$

where

$$S_1[\Psi](p^*) = \{(s, p, o, g) \mid (s, p, o, g) \in W_0, s = URI(p^*), g = URI(\Psi)\}$$

$$S_2[\Psi](p^*) = \{(s, p, o, g) \mid (s, p, o) \in \Psi[r^* \leftarrow p^*](\sigma_1), g = URI(\Psi)\}$$

It then follows that

$$q \in S_1[\Psi](p^*) \wedge q \notin S_2[\Psi](p^*)$$

By first conjunct, q can be derived from p^* using Ψ in σ_0 . By Lemma 1, the pivot tuple p^* and the rule Ψ are the unique way of deriving q in σ_0 . Furthermore, the subject of q is $URI(p^*)$. But, since $p^* \in A^-[\Psi](u)$, this derivation is invalidated by the deletion of the tuples in D from $R(\sigma_0)$. Hence, there would be no pivot tuple $t \in R^*(\sigma_0)$ and no rule $\Phi \in M$ that derives q in σ_0 , after the deletion of the tuples in D from $R(\sigma_0)$. In particular, there would be no pivot tuple $t \in R^*(\sigma_0)$ such that the subject of q is $URI(t)$, after the deletion of the tuples in D from $R(\sigma_0)$. Hence, if $q \in S_1[\Psi](p^*)$, then q has no derivation in σ_0 , after the deletion of the tuples in D from $R(\sigma_0)$.

However, after inserting the tuples in I into $R(\sigma_0)$, it might be the case that q could be derived from the pivot tuple p^* and the rule Ψ in σ_1 . But any such quad would be in $S_2[\Psi](p^*)$ and, therefore, removed from $S_1[\Psi](p^*)$.

Hence, if $q \in S_1[\Psi](p^*) \wedge q \notin S_2[\Psi](p^*)$, then q has no derivation in σ_0 , after the deletion of the tuples in D from $R(\sigma_0)$ and the insertion of the tuples in I

into $R(\sigma_0)$. But $R(\sigma_1) = (R(\sigma_0) \setminus D) \cup I$. Hence, if $q \in S_1[\Psi](p^*) \wedge q \notin S_2[\Psi](p^*)$, then q has no derivation in σ_1 , that is, $q \notin W_1$.

Thus, in either case, if $q \in \Delta^-(u)$ and $q \notin \Delta^+(u)$, then $q \notin W_1$.

Lemma 3 (Characterization of the Insertion Component).

$\Delta^+(u)$ is the set of quads whose derivation becomes valid in σ_1 due to some tuple $i \in I$.

Proof Sketch. Follows from Definitions 6 and 7.

Theorem 1 (Correctness of the Incremental Changesets).

$$W_1 = (W_0 \setminus \Delta^-(u)) \cup \Delta^+(u)$$

Proof. We will prove the set equality by showing mutual inclusion.

($\subseteq$) Assume that $q \in W_1$.

Case 1: Assume that $q \in W_0$.

Since $q \in W_1$, by Lemma 2, either $q \notin \Delta^-(u)$ or $q \in \Delta^+(u)$.

Assume $q \in \Delta^+(u)$. Then, trivially

$$q \in (W_0 \setminus \Delta^-(u)) \cup \Delta^+(u)$$

Assume $q \notin \Delta^+(u)$. Then, $q \notin \Delta^-(u)$. But, by assumption, $q \in W_0$. Hence, it follows that

$$q \in (W_0 \setminus \Delta^-(u))$$

Therefore, it follows that

$$q \in (W_0 \setminus \Delta^-(u)) \cup \Delta^+(u)$$

Case 2: Assume that $q \notin W_0$.

Then, since $q \in W_1$ but $q \notin W_0$, q became derivable in σ_1 due to some tuple $i \in I$. By Lemma 3, $q \in \Delta^+(u)$.

Therefore, in both cases, it follows that

$$q \in (W_0 \setminus \Delta^-(u)) \cup \Delta^+(u).$$

($\supseteq$) Assume that $q \in (W_0 \setminus \Delta^-(u)) \cup \Delta^+(u)$.

Case 1: Assume that $q \in \Delta^+(u)$.

Then, by Lemma 3, q is derivable under σ_1 , and thus $q \in W_1$.

Case 2: Assume that $q \in (W_0 \setminus \Delta^-(u))$.

Then, $q \in W_0$ and $q \notin \Delta^-(u)$. By Lemma 2, $q \in W_1$.

Therefore, in both cases, it follows that $q \in W_1$. □

Let $u = (D, I)$ be an update. Recall that σ_0 and σ_1 be the database states before and after applying u , and $W_1 = RDF_State(\mathcal{W}, \sigma_1)$ and $W_0 = RDF_State(\mathcal{W}, \sigma_0)$ are the view states before and after applying u . Let σ_D be the database state after applying the deletions in D , but before applying the insertions in I . Let $W_D = RDF_State(\mathcal{W}, \sigma_D)$ be the view state corresponding to W_D .

Corollary 1 (Minimality of the Incremental Changesets).

- (a) $\Delta^-(u) = (W_0 \setminus W_D)$
- (b) $(\Delta^+(u) \setminus W_D) = (W_1 \setminus W_D)$
- (c) If $(\Delta^+(u) \cap W_D) = \emptyset$, then $\Delta^+(u) = (W_1 \setminus W_D)$

Proof

First note that, executing $u = (D, I)$ on σ_0 is equivalent to executing the update $u_D = (D, \emptyset)$ on σ_0 , creating σ_D , and then executing the update $u_I = (\emptyset, I)$ on σ_D , creating σ_1 .

Part (a): By Theorem 1, since the insertion component of u_D is empty, we have

$$W_D = (W_0 \setminus \Delta^-(u))$$

Hence

$$(W_0 \setminus W_D) = (W_0 \setminus (W_0 \setminus \Delta^-(u)))$$

But $\Delta^-(u) \subseteq W_0$. Hence

$$(W_0 \setminus W_D) = \Delta^-(u)$$

Part (b): By Theorem 1, since the deletion component of u_I is empty, we have

$$W_1 = W_D \cup \Delta^+(u)$$

Hence

$$(W_1 \setminus W_D) = (W_D \cup \Delta^+(u)) \setminus W_D$$

Hence

$$(W_1 \setminus W_D) = (\Delta^+(u) \setminus W_D)$$

Part (c): First observe that, if $(\Delta^+(u) \cap W_D) = \emptyset$, then $(\Delta^+(u) \setminus W_D) = \Delta^+(u)$. Hence, from Part (b), it follows that

$$(W_1 \setminus W_D) = (\Delta^+(u) \setminus W_D) = \Delta^+(u)$$

□

References

1. Abo Khamis, M., Chmielewski, E., Draghici, A., Kara, A., Olteanu, D.: Maintaining queries under updates using heavy-light partitioning of the input relations. *Proc. ACM Manag. Data* **4**(2) (2026). <https://doi.org/10.1145/3801905>
2. Ali, M.A., Fernandes, A.A.A., Paton, N.W.: Movie: An incremental maintenance system for materialized object views. *Data Knowl. Eng.* **47**(2), 131–166 (Nov 2003)
3. Anonymized1: Anonymized
4. Anonymized6: Anonymized
5. Anonymized7: Anonymized
6. Augusto, C., Bertolino, A., Angelis, G.D., Lonetti, F., Morán, J.: Large language models for software testing: A research roadmap (2025), <https://arxiv.org/abs/2509.25043>
7. Battiston, I., Kathuria, K., Boncz, P.: Openivm: a sql-to-sql compiler for incremental computations. In: *Companion of the 2024 International Conference on Management of Data*. p. 516–519. ACM (Jun 2024). <https://doi.org/10.1145/3626246.3654743>
8. Beg, A., O’Donoghue, D., Monahan, R.: Leveraging llms for formal software requirements – challenges and prospects (2025), <https://arxiv.org/abs/2507.14330>
9. Budiu, M., Chajed, T., McSherry, F., Ryzhyk, L., Tannen, V.: Dbsp: Automatic incremental view maintenance for rich query languages. *Proc. VLDB Endow.* **16**(7), 1601–1614 (Mar 2023). <https://doi.org/10.14778/3587136.3587137>
10. Ceri, S., Widom, J.: Deriving production rules for incremental view maintenance. In: *Proceedings of the 17th International Conference on Very Large Data Bases*. pp. 577–589. VLDB ’91, San Francisco, CA, USA (1991)
11. Councilman, A., Fu, D.J., Gupta, A., Wang, C., Grove, D., Wang, Y.X., Adve, V.: Towards formal verification of llm-generated code from natural language prompts (2025), <https://arxiv.org/abs/2507.13290>
12. Ferrari, A., Spoletini, P.: Formal requirements engineering and large language models: A two-way roadmap. *Information and Software Technology* **181**, 107697 (2025). <https://doi.org/10.1016/j.infsof.2025.107697>
13. Han, S., Ives, Z.G.: Implementing views for property graphs. *SIGMOD Rec.* **54**(1), 59–68 (2025). <https://doi.org/10.1145/3733620.3733633>
14. Hu, X., Wang, Q.: Towards update-dependent analysis of query maintenance. *Proc. ACM Manag. Data* **3**(2) (2025). <https://doi.org/10.1145/3725254>
15. Jin, X., Liao, H.: An algorithm for incremental maintenance of materialized xpath view. In: *Proc. 11th International Conf. on Web-age Information Management*. pp. 513–524. WAIM’10, Springer-Verlag, Berlin, Heidelberg (2010)

16. Le-Cong, T., Le, B., Murray, T.: Can LLMs reason about program semantics? a comprehensive evaluation of LLMs on formal specification inference. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 21991–22014. Association for Computational Linguistics, Vienna, Austria (Jul 2025). <https://doi.org/10.18653/v1/2025.acl-long.1068>
17. Li, P., Glavic, B., Gawlick, D., Krishnaswamy, V., Liu, Z.H., Porobic, D., Niu, X.: In-memory incremental maintenance of provenance sketches. In: Proceedings 29th International Conference on Extending Database Technology, EDBT 2026, Tampere, Finland, March 24–27, 2026. pp. 56–68. OpenProceedings.org (2026). <https://doi.org/10.48786/EDBT.2026.05>
18. Liu, X., Shen, S., Li, B., Ma, P., Jiang, R., Zhang, Y., Fan, J., Li, G., Tang, N., Luo, Y.: A survey of text-to-sql in the era of llms: Where are we, and where are we going? *IEEE Transactions on Knowledge and Data Engineering* **37**(10), 5735–5754 (2025). <https://doi.org/10.1109/TKDE.2025.3592032>
19. Lukyanenko, R., et al.: Manifesto for software development and modeling in the age of artificial intelligence. *SSRN Electronic Journal* (12 2025). <https://doi.org/10.2139/ssrn.5881104>
20. Luo, Y., Li, G., Fan, J., Chai, C., Tang, N.: Natural language to sql: State of the art and open problems. *Proc. VLDB Endow.* **18**(12), 5466–5471 (Aug 2025). <https://doi.org/10.14778/3750601.3750696>
21. Mitchell, J., Kim, B.H., Zhou, C., Wang, C.: Understanding formal reasoning failures in llms as abstract interpreters. In: Proceedings of the 1st ACM SIGPLAN International Workshop on Language Models and Programming Languages. p. 71–83. *LMPL '25*, Association for Computing Machinery, New York, NY, USA (2025). <https://doi.org/10.1145/3759425.3763389>
22. Mitchell, J.L., Kim, B.H., Zhou, C., Wang, C.: Can llms formally reason as abstract interpreters for program analysis? *CoRR* **abs/2503.12686** (03 2025). <https://doi.org/10.48550/arXiv.2503.12686>
23. Motschnig-Pitrik, R.: The viewpoint abstraction in object-oriented modeling and the UML. In: Proceedings of the 19th International Conference on Conceptual Modeling. pp. 543–557. *ER'00*, Springer-Verlag, Berlin, Heidelberg (2000). https://doi.org/10.1007/3-540-45393-8_39
24. Murphy, W., Holzer, N., Qiao, F., Cui, L., Rothkopf, R., Koenig, N., Santolucito, M.: Combining llm code generation with formal specifications and reactive program synthesis (2024), <https://arxiv.org/abs/2410.19736>
25. Olteanu, D.: Recent increments in incremental view maintenance (2024), <https://arxiv.org/abs/2404.17679>
26. Poggi, A., Lembo, D., Calvanese, D., De Giacomo, G., Lenzerini, M., Rosati, R.: Linking data to ontologies. In: Spaccapietra, S. (ed.) *Journal on Data Semantics X*, pp. 133–173. Springer-Verlag, Berlin, Heidelberg (2008), <http://dl.acm.org/citation.cfm?id=1793934.1793939>
27. Sequeda, J.F., Arenas, M., Miranker, D.P.: OBDA: query rewriting or materialization? in practice, both! In: *The Semantic Web - ISWC 2014 - 13th International Semantic Web Conference*, October 19–23, 2014. Proceedings, Part I. pp. 535–551. Springer, Cham, Riva del Garda, Italy (2014). https://doi.org/10.1007/978-3-319-11964-9_34
28. Shi, L., Tang, Z., Zhang, N., Zhang, X., Yang, Z.: A survey on employing large language models for text-to-sql tasks. *ACM Comput. Surv.* (Jun 2025). <https://doi.org/10.1145/3737873>

29. Singh, A., Shetty, A., Ehtesham, A., Kumar, S., Khoei, T.T.: A survey of large language model-based generative AI for text-to-SQL: Benchmarks, applications, use cases, and challenges. In: 2025 IEEE 15th Annual Computing and Communication Workshop and Conference (CCWC). pp. 00015–00021 (2025). <https://doi.org/10.1109/CCWC62904.2025.10903689>
30. Singh, V., Cassel, D., Weir, N., Feng, N., Bayless, S.: Verge: Formal refinement and guidance engine for verifiable llm reasoning (2026), <https://arxiv.org/abs/2601.20055>
31. Staquet, D., Buelens, B., Van den Bussche, J.: Incremental view maintenance for sparql queries: Adapting the counting algorithm. *ACM Trans. Web* (Feb 2026). <https://doi.org/10.1145/3796549>
32. Svingos, C., Hernich, A., Gildhoff, H., Papakonstantinou, Y., Ioannidis, Y.: Foreign keys open the door for faster incremental view maintenance. *Proc. ACM Manag. Data* **1**(1) (2023). <https://doi.org/10.1145/3588720>
33. Swartz, A.: Musicbrainz: a semantic web service. *IEEE Intelligent Systems* **17**(1), 76–77 (2002). <https://doi.org/10.1109/5254.988466>
34. Vandenbrande, M., Taelman, R., Bonte, P., Ongena, F.: Incremunica: Web-based incremental view maintenance for sparql. In: Curry, E., Acosta, M., Poveda-Villalón, M., van Erp, M., Ojo, A., Hose, K., Shimizu, C., Lisen, P. (eds.) *The Semantic Web*. pp. 297–315. Springer Nature Switzerland, Cham (2025)
35. Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., Wang, Q.: Software testing with large language models: Survey, agents, and vision. *IEEE Trans. Softw. Eng.* **50**(4), 911–936 (Apr 2024). <https://doi.org/10.1109/TSE.2024.3368208>
36. Wang, K., Mao, B., Jia, S., Ding, Y., Han, D., Ma, T., Cao, B.: Sgcr: A specification-grounded framework for trustworthy llm code review (2026), <https://arxiv.org/abs/2512.17540>
37. Yadav, R., et al.: Enzyme: Incremental view maintenance for data engineering (2026), <https://arxiv.org/abs/2603.27775>
38. Zhang, Y., et al.: Position: Trustworthy AI agents require the integration of large language models and formal methods. In: *Proceedings of the 42nd International Conference on Machine Learning*. *Proceedings of Machine Learning Research*, vol. 267, pp. 82441–82459. PMLR (13–19 Jul 2025), <https://proceedings.mlr.press/v267/zhang25ds.html>
39. Zhao, W., Rusu, F., Dong, B., Wu, K., Nugent, P.: Incremental view maintenance over array data. In: *Proc. 2017 ACM International Conf. on Management of Data*. pp. 139–154. SIGMOD ’17, ACM, New York, NY, USA (2017)